

铁路智能代理安全评测软件源代码

软件名称：铁路智能代理安全评测软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. scripts/build_corpus.py
2. scripts/build_experiment_jobs.py
3. scripts/download_public_corpus.ps1
4. scripts/download_public_corpus.sh
5. scripts/evaluate_annotations.py
6. scripts/normalize_preannotations.py
7. scripts/prepare_preannotation_jobs.py
8. scripts/promote_reviewed_annotations.py
9. scripts/run_preannotation.py
10. scripts/run_qlora_inference.py
11. scripts/select_pilot_set.py
12. scripts/serve_review.py
13. scripts/train_qlora.py
14. scripts/validate_annotations.py
15. configs/risk_ontology.yaml
16. requirements.txt
17. schemas/preannotation_candidate.schema.json
18. schemas/risk_annotation.schema.json
19. tests/test_pipeline.py
20. tools/antiword/root/usr/share/antiword/8859-1.txt
21. tools/antiword/root/usr/share/antiword/8859-10.txt
22. tools/antiword/root/usr/share/antiword/8859-11.txt
23. tools/antiword/root/usr/share/antiword/8859-13.txt
24. tools/antiword/root/usr/share/antiword/8859-14.txt
25. tools/antiword/root/usr/share/antiword/8859-15.txt
26. tools/antiword/root/usr/share/antiword/8859-16.txt
27. tools/antiword/root/usr/share/antiword/8859-2.txt
28. tools/antiword/root/usr/share/antiword/8859-3.txt
29. tools/antiword/root/usr/share/antiword/8859-4.txt
30. tools/antiword/root/usr/share/antiword/8859-5.txt
31. tools/antiword/root/usr/share/antiword/8859-6.txt
32. tools/antiword/root/usr/share/antiword/8859-7.txt
33. tools/antiword/root/usr/share/antiword/8859-8.txt
34. tools/antiword/root/usr/share/antiword/8859-9.txt
35. tools/antiword/root/usr/share/antiword/cp1250.txt
36. tools/antiword/root/usr/share/antiword/cp1251.txt

scripts/build_corpus.py

```
#!/usr/bin/env python3
"""Inventory and extract the local safety corpus with source offsets."""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import os
import shutil
import subprocess
import tempfile
import zipfile
from pathlib import Path
from typing import Any, Iterable
from xml.etree import ElementTree

from pypdf import PdfReader

SUPPORTED_TEXT_FORMATS = {"pdf", "docx", "doc", "txt"}
WORD_NS = "{***}"

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(block)
    return digest.hexdigest()

def detect_format(path: Path) -> str:
    with path.open("rb") as stream:
        header = stream.read(16)
        if header.startswith(b"%PDF"):
            return "pdf"
        if header.startswith(b"\xd0\xcf\x11\xe0\xa1\xb1\xa1\xe1"):
            return "doc" if path.suffix.lower() in {".doc", ".docx"} else "ole"
        if header.startswith(b"PK"):
            try:
                with zipfile.ZipFile(path) as archive:
                    names = set(archive.namelist())
                    if "word/document.xml" in names:
                        return "docx"
                    if "xl/workbook.xml" in names:
                        return "xlsx"
                    if "ppt/presentation.xml" in names:
                        return "pptx"
                    return "zip"
            except zipfile.BadZipFile:
                return "invalid_zip"
        if header.startswith(b"{\\rtf"):
            return "rtf"
        if header.startswith(b"\xff\xd8\xff"):
            return "jpg"
    return path.suffix.lower().lstrip(".") or "unknown"

def language_hint(text: str) -> str:
    letters = sum(character.isalpha() for character in text)
    chinese = sum("\u4e00" <= character <= "\u9fff" for character in text)
    if not text.strip():
        return "unknown"
    if chinese >= 10 and chinese / max(letters, 1) >= 0.20:
        return "zh"
```

```

if letters >= 10:
    return "en"
return "unknown"

```

```

def make_segments(parts: Iterable[tuple[str, int | None, str]]) -> tuple[str, list[dict[str, Any]]]:
    full_text_parts: list[str] = []
    segments: list[dict[str, Any]] = []
    cursor = 0
    for segment_type, page, raw_text in parts:
        text = raw_text.strip()
        if not text:
            continue
        if full_text_parts:
            separator = "\n\n"
            full_text_parts.append(separator)
            cursor += len(separator)
        start = cursor
        full_text_parts.append(text)
        cursor += len(text)
        segments.append(
            {
                "segment_id": f"S{len(segments) + 1}",
                "segment_type": segment_type,
                "page": page,
                "start": start,
                "end": cursor,
                "text": text,
            }
        )
    return "".join(full_text_parts), segments

```

```

def extract_pdf(path: Path) -> tuple[str, list[dict[str, Any]]]:
    reader = PdfReader(str(path), strict=False)
    parts = (("page", page_number, page.extract_text() or "") for page_number, page in enumerate(reader.pages, 1))
    return make_segments(parts)

```

```

def extract_docx(path: Path) -> tuple[str, list[dict[str, Any]]]:
    with zipfile.ZipFile(path) as archive:
        xml_bytes = archive.read("word/document.xml")
    root = ElementTree.fromstring(xml_bytes)
    parts: list[tuple[str, int | None, str]] = []
    for paragraph in root.iter(f"{WORD_NS}p"):
        text_fragments: list[str] = []
        for node in paragraph.iter():
            if node.tag == f"{WORD_NS}t" and node.text:
                text_fragments.append(node.text)
            elif node.tag == f"{WORD_NS}tab":
                text_fragments.append("\t")
            elif node.tag in {f"{WORD_NS}br", f"{WORD_NS}cr"}:
                text_fragments.append("\n")
        text = "".join(text_fragments)
        if text.strip():
            parts.append(("paragraph", None, text))
    return make_segments(parts)

```

```

def extract_txt(path: Path) -> tuple[str, list[dict[str, Any]]]:
    raw = path.read_bytes()
    text = ""
    for encoding in ("utf-8-sig", "gb18030", "utf-16"):
        try:
            text = raw.decode(encoding)
            break
        except UnicodeDecodeError:
            continue
    if not text:
        text = raw.decode("utf-8", errors="replace")
    paragraphs = (part for part in text.replace("\r\n", "\n").split("\n\n"))

```

```

return make_segments(("paragraph", None, part) for part in paragraphs)

def extract_doc(path: Path) -> tuple[str, list[dict[str, Any]]]:
    project_root = Path(__file__).resolve().parents[1]
    private_antiword = project_root / "tools/antiword/root/usr/bin/antiword"
    private_catdoc = project_root / "tools/catdoc/root/usr/bin/catdoc"
    antiword = str(private_antiword) if private_antiword.exists() else shutil.which("antiword")
    catdoc = str(private_catdoc) if private_catdoc.exists() else shutil.which("catdoc")
    antiword_home = project_root / "tools/antiword/root/usr/share/antiword"
    libreoffice = shutil.which("libreoffice") or shutil.which("soffice")
    if antiword:
        result = subprocess.run(
            [antiword, "-m", "UTF-8.txt", str(path)],
            check=False,
            capture_output=True,
            env={**os.environ, "ANTIWORDHOME": str(antiword_home)},
        )
        if result.returncode == 0 and result.stdout.strip():
            text = result.stdout.decode("utf-8", errors="replace")
            return make_segments(("paragraph", None, part) for part in text.split("\n\n"))
    if catdoc:
        result = subprocess.run([catdoc, "-d", "UTF-8", str(path)], check=False, capture_output=True)
        if result.returncode == 0 and result.stdout.strip():
            text = result.stdout.decode("utf-8", errors="replace")
            return make_segments(("paragraph", None, part) for part in text.split("\n\n"))
        catdoc_error = result.stderr.decode("utf-8", errors="replace").strip()
    else:
        catdoc_error = ""
    if libreoffice:
        with tempfile.TemporaryDirectory(prefix="safety_doc_") as temporary_directory:
            subprocess.run(
                [libreoffice, "--headless", "--convert-to", "txt:Text", "--outdir", temporary_directory,
str(path)],
                check=True,
                capture_output=True,
            )
            converted = Path(temporary_directory) / f"{path.stem}.txt"
            if not converted.exists():
                raise RuntimeError("LibreOffice did not create a text file")
            return extract_txt(converted)
    if catdoc_error:
        raise RuntimeError(f"legacy_doc_extract_failed: {catdoc_error[:400]}")
    raise RuntimeError("tool_unavailable: install antiword, catdoc, or LibreOffice to extract legacy .doc files")

def extract_document(path: Path, actual_format: str) -> tuple[str, list[dict[str, Any]]]:
    if actual_format == "pdf":
        return extract_pdf(path)
    if actual_format == "docx":
        return extract_docx(path)
    if actual_format == "doc":
        return extract_doc(path)
    if actual_format == "txt":
        return extract_txt(path)
    raise RuntimeError(f"unsupported_format: {actual_format}")

def inventory_files(source_root: Path) -> list[Path]:
    return sorted(path for path in source_root.rglob("*") if path.is_file())

def build(args: argparse.Namespace) -> None:
    source_root = args.source_root.resolve()
    output_root = args.output_root.resolve()
    document_root = output_root / "documents"
    output_root.mkdir(parents=True, exist_ok=True)
    document_root.mkdir(parents=True, exist_ok=True)
    args.manifest.parent.mkdir(parents=True, exist_ok=True)

    paths = inventory_files(source_root)

```

```

if args.max_documents:
    paths = paths[: args.max_documents]

rows: list[dict[str, Any]] = []
first_by_hash: dict[str, str] = {}
for index, path in enumerate(paths, 1):
    relative_path = path.relative_to(source_root).as_posix()
    path_hash = hashlib.sha256(relative_path.encode("utf-8")).hexdigest()
    document_id = f"doc_{path_hash[:16]}"
    content_hash = sha256_file(path)
    duplicate_of = first_by_hash.get(content_hash, "")
    first_by_hash.setdefault(content_hash, document_id)
    actual_format = detect_format(path)
    row: dict[str, Any] = {
        "document_id": document_id,
        "source_group": relative_path.split("/", 1)[0],
        "relative_path": relative_path,
        "extension": path.suffix.lower().lstrip("."),
        "actual_format": actual_format,
        "size_bytes": path.stat().st_size,
        "sha256": content_hash,
        "duplicate_of": duplicate_of,
        "extract_status": "not_attempted",
        "language": "unknown",
        "segment_count": 0,
        "char_count": 0,
        "error": "",
    }
    if actual_format not in SUPPORTED_TEXT_FORMATS:
        row["extract_status"] = "unsupported"
        row["error"] = f"unsupported_format: {actual_format}"
    elif duplicate_of and not args.extract_duplicates:
        row["extract_status"] = "duplicate"
    else:
        try:
            text, segments = extract_document(path, actual_format)
            if not text.strip():
                raise RuntimeError("empty_text")
            language = language_hint(text)
            document = {
                "schema_version": "0.1.0",
                "document_id": document_id,
                "source_group": row["source_group"],
                "relative_path": relative_path,
                "sha256": content_hash,
                "actual_format": actual_format,
                "language": language,
                "text": text,
                "segments": segments,
            }
            output_path = document_root / f"{document_id}.json"
            output_path.write_text(json.dumps(document, ensure_ascii=False), encoding="utf-8")
            row.update(
                extract_status="success",
                language=language,
                segment_count=len(segments),
                char_count=len(text),
            )
        except Exception as error: # Keep one bad source from aborting the corpus build.
            row["extract_status"] = "failed"
            row["error"] = str(error).replace("\n", " ")[:500]
    rows.append(row)
    if index % 100 == 0 or index == len(paths):
        print(f"processed {index}/{len(paths)}")

fieldnames = list(rows[0].keys()) if rows else []
with args.manifest.open("w", encoding="utf-8-sig", newline="") as stream:
    writer = csv.DictWriter(stream, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)

```

```

status_counts: dict[str, int] = {}
for row in rows:
    status = str(row["extract_status"])
    status_counts[status] = status_counts.get(status, 0) + 1
print(json.dumps({"documents": len(rows), "status": status_counts}, ensure_ascii=False, indent=2))

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source-root", type=Path, default=Path("data/raw"))
    parser.add_argument("--output-root", type=Path, default=Path("data/processed/corpus"))
    parser.add_argument("--manifest", type=Path, default=Path("data/catalog/corpus_inventory.csv"))
    parser.add_argument("--max-documents", type=int, default=0)
    parser.add_argument("--extract-duplicates", action="store_true")
    return parser.parse_args()

if __name__ == "__main__":
    build(parse_args())

```

scripts/build_experiment_jobs.py

```

#!/usr/bin/env python3
"""Build split-specific teacher jobs for baseline and KG-constrained experiments."""

from __future__ import annotations

import argparse
import json
from collections import Counter
from pathlib import Path
from typing import Any

import yaml

def load_jsonl(path: Path) -> List[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def train_concept_context(mentions_path: Path, limit: int) -> str:
    counts: Counter[tuple[str, str]] = Counter()
    for mention in load_jsonl(mentions_path):
        if mention.get("split") == "train":
            counts[(mention.get("type", "UNKNOWN"), mention.get("normalized_name") or mention.get("text", ""))] += 1
    values = [f"{entity_type}: {name}" for (entity_type, name), _ in counts.most_common(limit) if name]
    return "; ".join(values)

def constraint_context(ontology: dict[str, Any], concepts: str) -> str:
    signatures = ontology.get("allowed_relation_signatures", {})
    lines = []
    for relation, signature in signatures.items():
        lines.append(
            f"{relation}: source={' '.join(signature.get('source', []))}; "
            f"target={' '.join(signature.get('target', []))}"
        )
    return (
        "\n\nKNOWLEDGE-GRAPH CONSTRAINTS: Use the following ontology signatures before proposing a relation. "
        "If a source/target type pair is illegal, omit that relation. Keep entity text as exact source spans. "
        "Known training concepts are retrieval hints only; do not copy a concept unless the supplied text supports "
        it.\n"
        "LEGAL RELATION SIGNATURES:\n"
        + "\n".join(lines)
        + "\n\nTRAINING CONCEPT HINTS:\n"
        + concepts
    )

```

```

def run(args: argparse.Namespace) -> int:
    jobs = load_jsonl(args.jobs)
    manifest = load_jsonl(args.manifest)
    split_by_document = {row["document_id"]: row["split"] for row in manifest}
    selected = [job for job in jobs if split_by_document.get(job["document_id"]) == args.split]
    if args.mode == "kg_constrained":
        ontology = yaml.safe_load(args.ontology.read_text(encoding="utf-8"))
        concepts = train_concept_context(args.mentions, args.concept_limit)
        suffix = constraint_context(ontology, concepts)
        for job in selected:
            job["system_instruction"] += suffix
            job["experiment_mode"] = args.mode
    else:
        for job in selected:
            job["experiment_mode"] = args.mode
    args.output.parent.mkdir(parents=True, exist_ok=True)
    with args.output.open("w", encoding="utf-8") as stream:
        for job in selected:
            stream.write(json.dumps(job, ensure_ascii=False) + "\n")
    print(json.dumps({"split": args.split, "mode": args.mode, "jobs": len(selected), "output": str(args.output)}),
    indent=2))
    return 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--jobs", type=Path, default=Path("data/processed/preannotation/jobs.jsonl"))
    parser.add_argument("--manifest", type=Path, default=Path("data/processed/reviewed/split_manifest.jsonl"))
    parser.add_argument("--mentions", type=Path,
    default=Path("data/processed/reviewed/knowledge_graph/mentions.jsonl"))
    parser.add_argument("--ontology", type=Path, default=Path("configs/risk_ontology.yaml"))
    parser.add_argument("--split", choices=["train", "validation", "test"], default="test")
    parser.add_argument("--mode", choices=["baseline", "kg_constrained"], default="baseline")
    parser.add_argument("--concept-limit", type=int, default=120)
    parser.add_argument("--output", type=Path, required=True)
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(run(parse_args()))

```

scripts/download_public_corpus.ps1

```

param(
    [int]$MaxReports = 0,
    [int]$DelayMilliseconds = 250
)

$ErrorActionPreference = "Stop"
$ProgressPreference = "SilentlyContinue"
$repoRoot = Split-Path -Parent $PSScriptRoot
$rawDir = Join-Path $repoRoot "data\raw\raib"
$catalogDir = Join-Path $repoRoot "data\catalog"
$null = New-Item -ItemType Directory -Force -Path $rawDir, $catalogDir

$headers = @{ "User-Agent" = "railway-agent-safety-security-research/0.1 (public research corpus; contact via repository)" }
$listUrl = ""
$reportUrls = [System.Collections.Generic.HashSet[string]]::new()

function Get-RemoteText([string]$Url) {
    (Invoke-WebRequest -Uri $Url -Headers $headers -UseBasicParsing).Content
}

Write-Host "Collecting public RAIB report pages..."
for ($page = 1; $page -le 20; $page++) {
    $url = if ($page -eq 1) { $listUrl } else { "${listUrl}?page=$page" }
    try {

```



```

$html = Get-RemoteText $url
$matches = [regex]::Matches($html, 'href="(\/raib-reports/[^\"]+)"')
foreach ($match in $matches) {
    $path = $match.Groups[1].Value
    if ($path -notmatch '/raib-reports\?$' -and $path -notmatch '/raib-reports/page' -and $path -notmatch
'/email-signup') {
        $null = $reportUrls.Add("****")
    }
}
} catch {
    Write-Warning "Unable to read list page $page: $($_.Exception.Message)"
}
Start-Sleep -Milliseconds $DelayMilliseconds
}

$orderedUrls = @($reportUrls | Sort-Object)
if ($MaxReports -gt 0) {
    $orderedUrls = @($orderedUrls | Select-Object -First $MaxReports)
}

$manifestPath = Join-Path $catalogDir "raib_manifest.csv"
if (-not (Test-Path $manifestPath)) {
    "report_url,pdf_url,file,status,error" | Set-Content -Encoding UTF8 $manifestPath
}

$counter = 0
foreach ($reportUrl in $orderedUrls) {
    $counter++
    try {
        $html = Get-RemoteText $reportUrl
        $pdfMatch = [regex]::Match($html, 'href="(****)"')
        if (-not $pdfMatch.Success) {
            "$reportUrl,,,no-pdf," | Add-Content -Encoding UTF8 $manifestPath
            continue
        }

        $pdfUrl = [System.Net.WebUtility]::HtmlDecode($pdfMatch.Groups[1].Value)
        $fileName = [System.IO.Path]::GetFileName(([System.Uri]$pdfUrl).AbsolutePath)
        $fileName = $fileName -replace '[^a-zA-Z0-9._-]', '_'
        $target = Join-Path $rawDir $fileName
        if (-not (Test-Path $target)) {
            Invoke-WebRequest -Uri $pdfUrl -Headers $headers -OutFile $target -UseBasicParsing
        }
        "$reportUrl,$pdfUrl,$fileName,downloaded," | Add-Content -Encoding UTF8 $manifestPath
        Write-Host "[$counter/$($orderedUrls.Count)] $fileName"
    } catch {
        $message = $_.Exception.Message -replace '[\r\n,]', ' '
        "$reportUrl,,,error,$message" | Add-Content -Encoding UTF8 $manifestPath
        Write-Warning "[$counter/$($orderedUrls.Count)] $reportUrl: $message"
    }
    Start-Sleep -Milliseconds $DelayMilliseconds
}

Write-Host "Finished. Report pages: $($orderedUrls.Count). Files: $rawDir. Manifest: $manifestPath"

```

scripts/download_public_corpus.sh

```

#!/usr/bin/env bash

set -u

start=1
count=25
delay_ms=250
while [ "$#" -gt 0 ]; do
    case "$1" in
        --start) start="$2"; shift 2 ;;
        --count) count="$2"; shift 2 ;;
        --delay-ms) delay_ms="$2"; shift 2 ;;
    esac
done

```

```

*) printf 'Unknown argument: %s\n' "$1" >&2; exit 2 ;;
esac
done

repo_root=$(cd "$(dirname "$0")/.." && pwd)
raw_dir="$repo_root/data/raw/raib"
manifest="$repo_root/data/catalog/raib_manifest.csv"
work_dir=$(mktemp -d "${TMPDIR:-/tmp}/raib-segment.XXXXXX")
trap 'rm -rf "$work_dir"' EXIT

mkdir -p "$raw_dir" "$(dirname "$manifest")"
ua='railway-agent-safety-security-research/0.1 (public research corpus)'
list_file="$repo_root/data/catalog/raib_report_urls.txt"

if [ ! -s "$list_file" ]; then
    list_file_tmp="$work_dir/report_urls"
    for page in $(seq 1 20); do
        list_url='***'
        [ "$page" -gt 1 ] && list_url="${list_url}?page=${page}"
        curl -L --fail --silent --show-error --connect-timeout 15 --max-time 45 \
            -A "$ua" "$list_url" -o "$work_dir/list-$page.html" || continue
        rg -o 'href="/raib-reports/[^"]+"' "$work_dir/list-$page.html" \
            | sed 's/^href="//; s/"$// ' \
            | sed '/\//email-signup$/d; /\//raib-reports\/page/d ' \
            | sed 's/^#*** >> "$list_file_tmp"
        sleep 0.25
    done
    sort -u "$list_file_tmp" > "$list_file"
fi

total=$(wc -l < "$list_file")
end=$((start + count - 1))
[ "$end" -gt "$total" ] && end="$total"
[ ! -f "$manifest" ] && printf 'report_url,pdf_url,file,status,error\n' > "$manifest"

for index in $(seq "$start" "$end"); do
    report_url=$(sed -n "${index}p" "$list_file")
    [ -n "$report_url" ] || continue
    if ! curl -L --fail --silent --show-error --connect-timeout 15 --max-time 45 \
        -A "$ua" "$report_url" -o "$work_dir/report.html"; then
        printf '%s,,error,report_page_fetch_failed\n' "$report_url" >> "$manifest"
        printf "[%s/%s] report page failed\n" "$index" "$total"
        continue
    fi

    pdf_url=$(rg -o 'href="*** \
        "$work_dir/report.html" | head -1 | sed 's/^href="//; s/"$// ' | sed 's/&\/&/g')
    if [ -z "$pdf_url" ]; then
        printf '%s,,no-pdf,\n' "$report_url" >> "$manifest"
        printf "[%s/%s] no PDF\n" "$index" "$total"
        continue
    fi

    file_name=$(basename "${pdf_url%{?}*}" | sed 's/[^a-zA-Z0-9._-]/_/g')
    target="$raw_dir/$file_name"
    status=already_present
    if [ ! -s "$target" ]; then
        if curl -L --fail --silent --show-error --connect-timeout 15 --max-time 90 \
            -A "$ua" "$pdf_url" -o "$target"; then
            status=downloaded
        else
            status=error
            printf '%s,,error,pdf_download_failed\n' "$report_url" >> "$manifest"
        fi
    fi

    if [ "$status" != error ]; then
        printf '%s,%s,%s,%s,\n' "$report_url" "$pdf_url" "$file_name" "$status" >> "$manifest"
    fi

    printf "[%s/%s] %s (%s)\n" "$index" "$total" "$file_name" "$status"
    sleep "0.$(printf '%03d' "$delay_ms")"
done

```

```
printf 'Segment finished: %s-%s of %s; PDF files: ' "$start" "$end" "$total"
find "$raw_dir" -maxdepth 1 -type f -name '*.pdf' | wc -l
```

scripts/evaluate_annotations.py

```
#!/usr/bin/env python3
"""Evaluate strict entity and relation extraction metrics for JSONL annotations."""

from __future__ import annotations

import argparse
import json
import re
from collections import Counter, defaultdict
from pathlib import Path
from typing import Any

def load_jsonl(path: Path) -> list[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def norm(value: str) -> str:
    return re.sub(r"\s+", " ", value).strip().casefold()

def unpack(rows: list[dict[str, Any]], index: list[dict[str, Any]] | None) -> dict[str, dict[str, Any]]:
    result = {}
    for position, row in enumerate(rows):
        if "annotation" in row:
            job_id = row["job_id"]
            annotation = row["annotation"]
        elif index:
            job_id = index[position]["job_id"]
            annotation = row
        else:
            job_id = f"record_{position + 1}"
            annotation = row
        result[job_id] = annotation
    return result

def entity_key(entity: dict[str, Any]) -> tuple[str, str]:
    return norm(entity["text"]), entity["type"]

def relation_key(relation: dict[str, Any], entities: dict[str, dict[str, Any]]) -> tuple[str, str, str]:
    source = entities.get(relation["source_id"], {})
    target = entities.get(relation["target_id"], {})
    return norm(source.get("text", "")), relation["type"], norm(target.get("text", ""))

def scores(gold: set[Any], predicted: set[Any]) -> dict[str, float]:
    true_positive = len(gold & predicted)
    precision = true_positive / len(predicted) if predicted else 0.0
    recall = true_positive / len(gold) if gold else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return {"precision": round(precision, 4), "recall": round(recall, 4), "f1": round(f1, 4), "gold": len(gold),
    "predicted": len(predicted), "correct": true_positive}

def aggregate(values: list[dict[str, float]]) -> dict[str, float]:
    gold = sum(value["gold"] for value in values)
    predicted = sum(value["predicted"] for value in values)
    correct = sum(value["correct"] for value in values)
    precision = correct / predicted if predicted else 0.0
    recall = correct / gold if gold else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
```

```

    return {"precision": round(precision, 4), "recall": round(recall, 4), "f1": round(f1, 4), "gold": gold,
"predicted": predicted, "correct": correct}

def run(args: argparse.Namespace) -> int:
    gold_rows = load_jsonl(args.gold)
    gold_index = load_jsonl(args.gold_index)
    predicted_rows = load_jsonl(args.predictions)
    gold = unpack(gold_rows, gold_index)
    predicted = unpack(predicted_rows, None)
    job_ids = sorted(set(gold) & set(predicted))
    entity_scores = []
    relation_scores = []
    claim_scores = []
    by_language: dict[str, list[str]] = defaultdict(list)
    for job_id in job_ids:
        g = gold[job_id]
        p = predicted[job_id]
        g_entities = {entity_key(entity) for entity in g.get("entities", [])}
        p_entities = {entity_key(entity) for entity in p.get("entities", [])}
        entity_scores.append(scores(g_entities, p_entities))
        g_by_id = {entity["id"]: entity for entity in g.get("entities", [])}
        p_by_id = {entity["id"]: entity for entity in p.get("entities", [])}
        g_relations = {relation_key(relation, g_by_id) for relation in g.get("relations", [])}
        p_relations = {relation_key(relation, p_by_id) for relation in p.get("relations", [])}
        relation_scores.append(scores(g_relations, p_relations))
        g_claims = {(relation_key(r, g_by_id), r["claim_status"]) for r in g.get("relations", [])}
        p_claims = {(relation_key(r, p_by_id), r["claim_status"]) for r in p.get("relations", [])}
        claim_scores.append(scores(g_claims, p_claims))
        by_language[g.get("language", "unknown")].append(job_id)

    def average(values: list[dict[str, float]]) -> dict[str, float]:
        if not values:
            return {"precision": 0.0, "recall": 0.0, "f1": 0.0, "gold": 0, "predicted": 0, "correct": 0}
        return {key: round(sum(value[key] for value in values) / len(values), 4) for key in ("precision", "recall",
"f1")}

    result: dict[str, Any] = {
        "jobs_gold": len(gold),
        "jobs_predicted": len(predicted),
        "jobs_evaluated": len(job_ids),
        "entity_strict": aggregate(entity_scores),
        "relation_strict": aggregate(relation_scores),
        "relation_with_claim_status": aggregate(claim_scores),
        "macro_by_job": {"entity_strict": average(entity_scores), "relation_strict": average(relation_scores),
"relation_with_claim_status": average(claim_scores)},
        "by_language": {},
    }
    for language, language_jobs in by_language.items():
        language_entity = [entity_scores[job_ids.index(job_id)] for job_id in language_jobs]
        language_relation = [relation_scores[job_ids.index(job_id)] for job_id in language_jobs]
        result["by_language"][language] = {"jobs": len(language_jobs), "entity_strict": average(language_entity),
"relation_strict": average(language_relation)}
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(json.dumps(result, ensure_ascii=False, indent=2))
    return 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--gold", type=Path, required=True)
    parser.add_argument("--gold-index", type=Path, default=Path("data/processed/reviewed/gold/record_index.jsonl"))
    parser.add_argument("--predictions", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(run(parse_args()))

```

scripts/normalize_preannotations.py

```
#!/usr/bin/env python3
"""Resolve teacher evidence quotes to deterministic source offsets and pages."""

from __future__ import annotations

import argparse
import json
import re
from pathlib import Path
from typing import Any

import yaml

def load_jsonl(path: Path) -> List[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def flexible_span(source: str, value: str) -> tuple[int, int]:
    exact_start = source.find(value)
    if exact_start >= 0:
        if source.find(value, exact_start + 1) >= 0:
            raise ValueError(f"ambiguous repeated text: {value[:80]!r}")
        return exact_start, exact_start + len(value)
    tokens = re.split(r"\s+", value.strip())
    if not tokens or not all(tokens):
        raise ValueError("empty text")
    pattern = r"\s+".join(re.escape(token) for token in tokens)
    matches = list(re.finditer(pattern, source))
    if len(matches) != 1:
        reason = "not found" if not matches else "ambiguous"
        raise ValueError(f"{reason} after whitespace normalization: {value[:80]!r}")
    return matches[0].start(), matches[0].end()

def resolve_evidence(evidence: dict[str, Any], segment_by_id: dict[str, dict[str, Any]]) -> dict[str, Any]:
    segment = segment_by_id.get(evidence["segment_id"])
    if not segment:
        raise ValueError(f"unknown segment_id: {evidence['segment_id']}")
    quote = evidence["text"]
    try:
        local_start, local_end = flexible_span(segment["text"], quote)
    except ValueError as error:
        raise ValueError(f"invalid evidence in {evidence['segment_id']}: {error}") from error
    start = segment["start"] + local_start
    return {
        "text": segment["text"][local_start:local_end],
        "segment_id": evidence["segment_id"],
        "page": segment.get("page"),
        "start": start,
        "end": segment["start"] + local_end,
    }

def normalize(args: argparse.Namespace) -> int:
    jobs = {job["job_id"]: job for job in load_jsonl(args.jobs)}
    candidates = load_jsonl(args.candidates)
    ontology = yaml.safe_load(args.ontology.read_text(encoding="utf-8"))
    signatures = ontology.get("allowed_relation_signatures", {})
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.errors.parent.mkdir(parents=True, exist_ok=True)
    successes = 0
    failures = 0
    invalid_candidates = 0
    with args.output.open("w", encoding="utf-8") as output_stream, args.errors.open("w", encoding="utf-8") as error_stream:
        for item in candidates:
            job_id = item["job_id"]
```

```

candidate = item["annotation"]
job = jobs.get(job_id)
if not job:
    error_stream.write(json.dumps({"job_id": job_id, "error": "job not found"}) + "\n")
    failures += 1
    continue
segment_by_id = {segment["segment_id"]: segment for segment in job["segments"]}
candidate_errors: list[str] = []
try:
    annotation = dict(candidate)
    normalized_entities: list[dict[str, Any]] = []
    valid_entity_ids: set[str] = set()
    for entity in candidate["entities"]:
        try:
            evidence = resolve_evidence(entity["evidence"], segment_by_id)
            local_start, local_end = flexible_span(evidence["text"], entity["text"])
            source_entity_text = evidence["text"][local_start:local_end]
            normalized_entity = {**entity, "text": source_entity_text, "evidence": evidence}
            if source_entity_text != entity["text"]:
                normalized_entity["normalized_name"] = entity["text"]
            normalized_entities.append(normalized_entity)
            valid_entity_ids.add(entity["id"])
        except Exception as error:
            candidate_errors.append(f"entity {entity.get('id')}: {error}")
    normalized_relations: list[dict[str, Any]] = []
    for relation in candidate["relations"]:
        try:
            if relation["source_id"] not in valid_entity_ids or relation["target_id"] not in
valid_entity_ids:
                raise ValueError("source or target entity was rejected")
            normalized_relations.append(
                {
                    **relation,
                    "evidence": [
                        resolve_evidence(evidence, segment_by_id) for evidence in relation["evidence"]
                    ],
                }
            )
        except Exception as error:
            candidate_errors.append(f"relation {relation.get('id')}: {error}")
    annotation["entities"] = normalized_entities
    entity_by_id = {entity["id"]: entity for entity in normalized_entities}
    annotation["relations"] = normalized_relations
    constrained_relations: list[dict[str, Any]] = []
    for relation in normalized_relations:
        signature = signatures.get(relation["type"])
        source_type = entity_by_id[relation["source_id"]]["type"]
        target_type = entity_by_id[relation["target_id"]]["type"]
        if signature and (
            source_type not in signature.get("source", [])
            or target_type not in signature.get("target", [])
        ):
            candidate_errors.append(
                f"relation {relation['id']}: illegal signature {source_type} -{relation['type']}->
{target_type}"
            )
            continue
        constrained_relations.append(relation)
    annotation["relations"] = constrained_relations
    output_record: dict[str, Any] = {"job_id": job_id, "annotation": annotation} if args.include_job_id
else annotation
    output_stream.write(json.dumps(output_record, ensure_ascii=False) + "\n")
    successes += 1
    if candidate_errors:
        invalid_candidates += len(candidate_errors)
        error_stream.write(
            json.dumps({"job_id": job_id, "candidate_errors": candidate_errors}, ensure_ascii=False) +
"\n"
        )
except Exception as error:
    error_stream.write(json.dumps({"job_id": job_id, "error": str(error)}, ensure_ascii=False) + "\n")

```

```

        failures += 1
    print(json.dumps({"normalized": successes, "failed": failures, "invalid_candidates": invalid_candidates},
indent=2))
    return 1 if failures else 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--jobs", type=Path, default=Path("data/processed/preannotation/jobs.jsonl"))
    parser.add_argument("--candidates", type=Path, default=Path("data/processed/preannotation/candidates.jsonl"))
    parser.add_argument("--output", type=Path, default=Path("data/processed/preannotation/normalized.jsonl"))
    parser.add_argument("--errors", type=Path, default=Path("outputs/preannotation_normalization_errors.jsonl"))
    parser.add_argument("--ontology", type=Path, default=Path("configs/risk_ontology.yaml"))
    parser.add_argument("--include-job-id", action="store_true", help="Write {job_id, annotation} envelopes for
experiment alignment")
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(normalize(parse_args()))

```

scripts/prepare_preannotation_jobs.py

```

#!/usr/bin/env python3
"""Prepare provider-neutral JSONL jobs for teacher-model pre-annotation."""

from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path
from typing import Any

import yaml

SYSTEM_INSTRUCTION = """You are pre-annotating safety-critical documents. Extract only claims supported by the
supplied text. Do not invent missing causes, controls, regulations, or consequences. Return one complete JSON
object conforming to preannotation_candidate.schema.json. Entity IDs must be E1, E2, ...; relation IDs must be R1,
R2, .... Each entity text must be an exact, contiguous source substring inside its evidence text; never paraphrase,
expand, normalize, or add punctuation to an entity. For evidence, copy an exact, contiguous quote from one supplied
segment and return only its segment_id and text; do not calculate offsets or page numbers. Set entity and relation
review_status to pending, document review.status to unreviewed, and created_by to the supplied teacher_model.
Distinguish explicit, inferred, normative, and uncertain relations."""

def chunk_segments(segments: list[dict[str, Any]], max_characters: int) -> list[list[dict[str, Any]]]:
    chunks: list[list[dict[str, Any]]] = []
    current: list[dict[str, Any]] = []
    current_size = 0
    for segment in segments:
        segment_size = len(segment["text"])
        if current and current_size + segment_size > max_characters:
            chunks.append(current)
            current = []
            current_size = 0
        if segment_size > max_characters:
            text = segment["text"]
            for local_start in range(0, len(text), max_characters):
                piece = dict(segment)
                piece["segment_id"] = f"{segment['segment_id']}.{local_start // max_characters + 1}"
                piece["text"] = text[local_start : local_start + max_characters]
                piece["start"] = segment["start"] + local_start
                piece["end"] = piece["start"] + len(piece["text"])
            if current:
                chunks.append(current)
                current = []
                current_size = 0

```

```

        chunks.append([piece])
        current_size = 0
        continue
    current.append(segment)
    current_size += segment_size
if current:
    chunks.append(current)
return chunks

def prepare(args: argparse.Namespace) -> None:
    ontology = yaml.safe_load(args.ontology.read_text(encoding="utf-8"))
    with args.pilot_set.open(encoding="utf-8-sig", newline="") as stream:
        pilot_rows = list(csv.DictReader(stream))

    compact_ontology = {
        "entity_types": {key: value["description"] for key, value in ontology["entity_types"].items()},
        "relation_types": {key: value["description"] for key, value in ontology["relation_types"].items()},
        "claim_statuses": ontology["claim_statuses"],
    }
    args.output.parent.mkdir(parents=True, exist_ok=True)
    job_count = 0
    with args.output.open("w", encoding="utf-8") as stream:
        for row in pilot_rows:
            document_path = args.document_root / f"{row['document_id']}.json"
            document = json.loads(document_path.read_text(encoding="utf-8"))
            chunks = chunk_segments(document["segments"], args.max_characters)
            for chunk_number, chunk in enumerate(chunks, 1):
                job_count += 1
                job = {
                    "job_id": f"{row['document_id']}_C{chunk_number}",
                    "document_id": row["document_id"],
                    "language": document["language"],
                    "category": row["category"],
                    "source_path": document["relative_path"],
                    "teacher_model": args.teacher_model,
                    "system_instruction": SYSTEM_INSTRUCTION,
                    "ontology": compact_ontology,
                    "segments": chunk,
                    "expected_output_schema": "schemas/preannotation_candidate.schema.json",
                    "status": "pending",
                }
                stream.write(json.dumps(job, ensure_ascii=False) + "\n")
    print(json.dumps({"documents": len(pilot_rows), "jobs": job_count, "output": str(args.output)}, indent=2))

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--pilot-set", type=Path, default=Path("data/catalog/pilot_set.csv"))
    parser.add_argument("--document-root", type=Path, default=Path("data/processed/corpus/documents"))
    parser.add_argument("--ontology", type=Path, default=Path("configs/risk_ontology.yaml"))
    parser.add_argument("--output", type=Path, default=Path("data/processed/preannotation/jobs.jsonl"))
    parser.add_argument("--max-characters", type=int, default=12000)
    parser.add_argument("--teacher-model", default="unspecified-teacher")
    return parser.parse_args()

if __name__ == "__main__":
    prepare(parse_args())

```

scripts/promote_reviewed_annotations.py

```

#!/usr/bin/env python3
"""Promote screened annotations into gold splits and a provenance-bearing graph."""

from __future__ import annotations

import argparse
import csv

```



```

import hashlib
import json
import re
from collections import Counter, defaultdict
from pathlib import Path
from typing import Any

import yaml
from jsonschema import Draft202012Validator

def load_jsonl(path: Path) -> List[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def canonical_text(value: str) -> str:
    return re.sub(r"\s+", " ", value).strip().casefold()

def concept_id(language: str, entity_type: str, name: str) -> str:
    key = f"{language}|{entity_type}|{canonical_text(name)}".encode("utf-8")
    return "concept_" + hashlib.sha1(key).hexdigest()[:16]

def stable_rank(document_id: str) -> str:
    return hashlib.sha256(document_id.encode("utf-8")).hexdigest()

def choose_splits(documents: List[dict[str, str]]) -> dict[str, str]:
    """Choose a reproducible 20/4/4 document split, balanced by source group."""
    by_group: dict[str, List[dict[str, str]]] = defaultdict(list)
    for row in documents:
        by_group[row["source_group"]].append(row)
    assignment: dict[str, str] = {}
    for group, rows in sorted(by_group.items()):
        ordered = sorted(rows, key=lambda row: stable_rank(row["document_id"]))
        test_count = max(1, round(len(ordered) * 0.15))
        validation_count = max(1, round(len(ordered) * 0.15))
        for row in ordered[:test_count]:
            assignment[row["document_id"]] = "test"
        for row in ordered[test_count : test_count + validation_count]:
            assignment[row["document_id"]] = "validation"
        for row in ordered[test_count + validation_count :]:
            assignment[row["document_id"]] = "train"
    return assignment

def promote_annotation(annotation: dict[str, Any]) -> dict[str, Any]:
    promoted = json.loads(json.dumps(annotation, ensure_ascii=False))
    promoted["review"] = {
        "status": "single_reviewed",
        "reviewers": ["human_screened"],
        "notes": "Bulk accepted after human screening of the pre-annotation review set.",
    }
    for entity in promoted["entities"]:
        entity["review_status"] = "accepted"
    for relation in promoted["relations"]:
        relation["review_status"] = "accepted"
    return promoted

def read_pilot(path: Path) -> dict[str, dict[str, str]]:
    with path.open(encoding="utf-8-sig", newline="") as stream:
        return {row["document_id"]: row for row in csv.DictReader(stream)}

def validate_annotations(annotations: List[dict[str, Any]], schema_path: Path) -> None:
    schema = json.loads(schema_path.read_text(encoding="utf-8"))
    validator = Draft202012Validator(schema)
    errors = [error.message for annotation in annotations for error in validator.iter_errors(annotation)]
    if errors:

```

```
raise ValueError("promoted annotations failed schema validation: " + "; ".join(errors[:10]))
```

```
def build_graph(
    annotations: list[dict[str, Any]],
    jobs_by_id: dict[str, dict[str, Any]],
    split_by_document: dict[str, str],
    output_dir: Path,
) -> dict[str, int]:
    concepts: dict[str, dict[str, Any]] = {}
    mention_rows: list[dict[str, Any]] = []
    relation_rows: list[dict[str, Any]] = []
    for annotation in annotations:
        job_id = annotation["_job_id"]
        job = jobs_by_id[job_id]
        document_id = annotation["document_id"]
        split = split_by_document[document_id]
        mention_by_source_id: dict[str, str] = {}
        for entity in annotation["entities"]:
            name = entity.get("normalized_name") or entity["text"]
            cid = concept_id(annotation["language"], entity["type"], name)
            mention_id = f"{job_id}:{entity['id']}"
            mention_by_source_id[entity["id"]] = mention_id
            concepts.setdefault(
                cid,
                {
                    "concept_id": cid,
                    "canonical_name": name,
                    "language": annotation["language"],
                    "type": entity["type"],
                    "mention_count": 0,
                    "source_documents": [],
                },
            )
        concept = concepts[cid]
        concept["mention_count"] += 1
        if document_id not in concept["source_documents"]:
            concept["source_documents"].append(document_id)
        mention_rows.append(
            {
                "mention_id": mention_id,
                "concept_id": cid,
                "document_id": document_id,
                "job_id": job_id,
                "split": split,
                "source_id": entity["id"],
                "text": entity["text"],
                "type": entity["type"],
                "normalized_name": entity.get("normalized_name"),
                "confidence": entity["confidence"],
                "evidence": entity["evidence"],
                "provenance": {"source_path": job["source_path"], "teacher_model": entity.get("created_by")},
            }
        )
    for relation in annotation["relations"]:
        source_mention = mention_by_source_id[relation["source_id"]]
        target_mention = mention_by_source_id[relation["target_id"]]
        source_concept = next(row["concept_id"] for row in mention_rows if row["mention_id"] == source_mention)
        target_concept = next(row["concept_id"] for row in mention_rows if row["mention_id"] == target_mention)
        relation_rows.append(
            {
                "relation_id": f"{job_id}:{relation['id']}",
                "source_concept_id": source_concept,
                "target_concept_id": target_concept,
                "source_mention_id": source_mention,
                "target_mention_id": target_mention,
                "document_id": document_id,
                "job_id": job_id,
                "split": split,
                "type": relation["type"],
                "claim_status": relation["claim_status"],
            }
        )
```

```

        "confidence": relation["confidence"],
        "evidence": relation["evidence"],
        "provenance": {"source_path": job["source_path"], "teacher_model": relation.get("created_by")},
    }
)
output_dir.mkdir(parents=True, exist_ok=True)
write_jsonl(output_dir / "concepts.jsonl", list(concepts.values()))
write_jsonl(output_dir / "mentions.jsonl", mention_rows)
write_jsonl(output_dir / "relations.jsonl", relation_rows)
return {"concepts": len(concepts), "mentions": len(mention_rows), "relations": len(relation_rows)}

def write_jsonl(path: Path, rows: list[dict[str, Any]]) -> None:
    with path.open("w", encoding="utf-8") as stream:
        for row in rows:
            stream.write(json.dumps(row, ensure_ascii=False) + "\n")

def run(args: argparse.Namespace) -> int:
    normalized = load_jsonl(args.annotations)
    jobs = load_jsonl(args.jobs)
    candidates = load_jsonl(args.candidates) if args.candidates.exists() else []
    jobs_by_id = {job["job_id"]: job for job in jobs}
    candidate_job_ids = [row["job_id"] for row in candidates]
    if len(normalized) != len(jobs):
        raise ValueError(f"expected one normalized record per job: {len(normalized)} vs {len(jobs)}")
    if candidate_job_ids and set(candidate_job_ids) != set(jobs_by_id):
        raise ValueError("candidate job IDs do not exactly match jobs; refuse ambiguous block-to-job mapping")
    enriched = []
    for index, annotation in enumerate(normalized):
        job_id = candidate_job_ids[index] if candidate_job_ids else jobs[index]["job_id"]
        if annotation["document_id"] != jobs_by_id[job_id]["document_id"]:
            raise ValueError(f"document mismatch for {job_id}")
        item = promote_annotation(annotation)
        item["_job_id"] = job_id
        enriched.append(item)
    validate_annotations([{"key": value for key, value in annotation.items() if not key.startswith("_")} for
annotation in enriched], args.schema)

    pilot = read_pilot(args.pilot_set)
    documents = [{"document_id": doc_id, "source_group": row["source_group"], "category": row["category"],
"relative_path": row["relative_path"]} for doc_id, row in pilot.items()]
    split_by_document = choose_splits(documents)
    output_root = args.output
    gold_dir = output_root / "gold"
    gold_dir.mkdir(parents=True, exist_ok=True)
    write_jsonl(gold_dir / "all.jsonl", [{"k": v for k, v in annotation.items() if not k.startswith("_")} for
annotation in enriched])
    split_rows: dict[str, list[dict[str, Any]]] = defaultdict(list)
    for annotation in enriched:
        split_rows[split_by_document[annotation["document_id"]]].append({"k": v for k, v in annotation.items() if not
k.startswith("_")})
    for split, rows in split_rows.items():
        write_jsonl(gold_dir / f"{split}.jsonl", rows)
        write_jsonl(
            gold_dir / f"{split}_index.jsonl",
            [
                {
                    "record_index": index,
                    "job_id": annotation["_job_id"],
                    "document_id": annotation["document_id"],
                    "split": split,
                }
                for index, annotation in enumerate(
                    [item for item in enriched if split_by_document[item["document_id"]] == split]
                )
            ],
        )
    write_jsonl(
        gold_dir / "record_index.jsonl",
        [

```

```

        {
            "record_index": index,
            "job_id": annotation["_job_id"],
            "document_id": annotation["document_id"],
            "split": split_by_document[annotation["document_id"]],
        }
    for index, annotation in enumerate(enriched)
],
)
manifest = []
for row in sorted(documents, key=lambda item: item["document_id"]):
    manifest.append({"row": row, "split": split_by_document[row["document_id"]]})
write_jsonl(output_root / "split_manifest.jsonl", manifest)
graph_stats = build_graph(enriched, jobs_by_id, split_by_document, output_root / "knowledge_graph")
summary = {
    "jobs": len(enriched),
    "documents": len(split_by_document),
    "split_documents": dict(Counter(split_by_document.values())),
    "split_chunks": dict(Counter(split_by_document[a["document_id"] for a in enriched])),
    "entities": sum(len(a["entities"]) for a in enriched),
    "relations": sum(len(a["relations"]) for a in enriched),
    "graph": graph_stats,
    "review_policy": "bulk_accepted_after_human_screening",
    "schema_version": "0.1.0",
}
(output_root / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2) + "\n",
encoding="utf-8")
print(json.dumps(summary, ensure_ascii=False, indent=2))
return 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--annotations", type=Path,
default=Path("data/processed/preannotation/sub2api_terra_normalized.jsonl"))
    parser.add_argument("--jobs", type=Path, default=Path("data/processed/preannotation/jobs.jsonl"))
    parser.add_argument("--candidates", type=Path,
default=Path("data/processed/preannotation/sub2api_terra_candidates.jsonl"))
    parser.add_argument("--pilot-set", type=Path, default=Path("data/catalog/pilot_set.csv"))
    parser.add_argument("--schema", type=Path, default=Path("schemas/risk_annotation.schema.json"))
    parser.add_argument("--output", type=Path, default=Path("data/processed/reviewed"))
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(run(parse_args()))

```

scripts/run_preannotation.py

```

#!/usr/bin/env python3
"""Run provider-neutral pre-annotation jobs through Ollama or OpenAI."""

from __future__ import annotations

import argparse
import json
import os
import time
import urllib.request
from pathlib import Path
from typing import Any

from jsonschema import Draft202012Validator

def load_jobs(path: Path) -> list[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

```

```

def user_payload(job: dict[str, Any]) -> str:
    payload = {
        "document_id": job["document_id"],
        "language": job["language"],
        "teacher_model": job["teacher_model"],
        "ontology": job["ontology"],
        "segments": job["segments"],
    }
    return json.dumps(payload, ensure_ascii=False)

def run_ollama(job: dict[str, Any], model: str, schema: dict[str, Any], timeout: int, think: bool) -> tuple[str, Any]:
    body = {
        "model": model,
        "stream": False,
        "think": think,
        "format": schema,
        "messages": [
            {"role": "system", "content": job["system_instruction"]},
            {"role": "user", "content": user_payload(job)},
        ],
        "options": {"temperature": 0, "seed": 42},
    }
    request = urllib.request.Request(
        """
        data=json.dumps(body).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(request, timeout=timeout) as response:
        envelope = json.loads(response.read().decode("utf-8"))
    return envelope["message"]["content"], {
        "prompt_eval_count": envelope.get("prompt_eval_count"),
        "eval_count": envelope.get("eval_count"),
        "total_duration": envelope.get("total_duration"),
    }

def run_openai(
    job: dict[str, Any],
    model: str,
    schema: dict[str, Any],
    timeout: int,
    _: bool,
    base_url: str | None,
    api_key_env: str,
) -> tuple[str, Any]:
    from openai import OpenAI

    api_key = os.environ.get(api_key_env)
    if not api_key:
        raise RuntimeError(f"missing API key environment variable: {api_key_env}")
    client_kwargs: dict[str, Any] = {"api_key": api_key, "timeout": timeout}
    if base_url:
        client_kwargs["base_url"] = base_url.rstrip("/")
    client = OpenAI(**client_kwargs)
    response = client.responses.create(
        model=model,
        instructions=job["system_instruction"],
        input=user_payload(job),
        text={
            "format": {
                "type": "json_schema",
                "name": "safety_risk_preannotation",
                "strict": False,
                "schema": schema,
            }
        },
        reasoning={"effort": "low"},
        store=False,
    )

```

```

        max_output_tokens=12000,
    )
    usage = response.usage.model_dump() if response.usage else None
    return response.output_text, usage

def run(args: argparse.Namespace) -> int:
    schema = json.loads(args.schema.read_text(encoding="utf-8"))
    validator = Draft202012Validator(schema)
    jobs = load_jobs(args.jobs)
    existing_ids: set[str] = set()
    if args.output.exists() and args.resume:
        for line in args.output.read_text(encoding="utf-8").splitlines():
            if line.strip():
                existing_ids.add(json.loads(line)["job_id"])
    pending = [job for job in jobs if job["job_id"] not in existing_ids]
    if args.job_id:
        pending = [job for job in pending if job["job_id"] == args.job_id]
        if not pending:
            raise SystemExit(f"job not found or already completed: {args.job_id}")
    if args.max_jobs:
        pending = pending[: args.max_jobs]

    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.log.parent.mkdir(parents=True, exist_ok=True)
    provider_runner = run_ollama if args.provider == "ollama" else run_openai
    failures = 0
    with args.output.open("a", encoding="utf-8") as output_stream, args.log.open("a", encoding="utf-8") as log_stream:
        for index, job in enumerate(pending, 1):
            started = time.monotonic()
            log_record: dict[str, Any] = {
                "job_id": job["job_id"],
                "provider": args.provider,
                "model": args.model,
            }
            try:
                call_job = job
                if args.repair_schema:
                    call_job = dict(job)
                    call_job["system_instruction"] = (
                        job["system_instruction"]
                        + "\n\nREPAIR CONSTRAINTS: Keep the exact schema field meanings. "
                        + "Use only the listed relation_types as the relation 'type'. "
                        + "Use only explicit, inferred, normative, or uncertain as relation claim_status. "
                        + "Use only pending for entity/relation review_status and unreviewed for review.status. "
                        + "Never put a relation type in claim_status or review.status. "
                        + "If a claim cannot be represented by the listed ontology, omit it."
                    )
                last_error: Exception | None = None
                for attempt in range(args.retries + 1):
                    try:
                        if args.provider == "openai":
                            content, usage = provider_runner(
                                call_job,
                                args.model,
                                schema,
                                args.timeout,
                                args.think,
                                args.base_url,
                                args.api_key_env,
                            )
                        else:
                            content, usage = provider_runner(call_job, args.model, schema, args.timeout,
                                args.think)
                    except Exception as error:
                        last_error = error
                        transient = any(token in str(error).lower() for token in (" 429", " 500", " 502", " 503",
                            "timeout", "temporarily unavailable"))
                        if not transient or attempt >= args.retries:

```

```

        raise
        time.sleep(min(30, 5 * (attempt + 1)))
    else:
        raise last_error or RuntimeError("provider call failed")
    annotation = json.loads(content)
    annotation.setdefault("schema_version", "0.1.0")
    annotation.setdefault("document_id", job["document_id"])
    annotation.setdefault("language", job["language"])
    annotation.setdefault("review", {"status": "unreviewed", "reviewers": [], "notes": "teacher pre-
annotation"})
    for entity in annotation.get("entities", []):
        entity.setdefault("created_by", job["teacher_model"])
        entity.setdefault("review_status", "pending")
    for relation in annotation.get("relations", []):
        relation.setdefault("created_by", job["teacher_model"])
        relation.setdefault("review_status", "pending")
    schema_errors = [error.message for error in validator.iter_errors(annotation)]
    if annotation.get("document_id") != job["document_id"]:
        schema_errors.append("document_id does not match job")
    if schema_errors:
        raise ValueError("; ".join(schema_errors[:10]))
    output_stream.write(json.dumps({"job_id": job["job_id"], "annotation": annotation},
ensure_ascii=False) + "\n")
    output_stream.flush()
    log_record.update(status="success", usage=usage)
except Exception as error:
    failures += 1
    log_record.update(status="failed", error=str(error)[:2000])
    log_record["elapsed_seconds"] = round(time.monotonic() - started, 3)
    log_stream.write(json.dumps(log_record, ensure_ascii=False) + "\n")
    log_stream.flush()
    print(f"{index}/{len(pending)} {job['job_id']} {log_record['status']}")
return 1 if failures else 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--provider", choices=["ollama", "openai"], required=True)
    parser.add_argument("--model", required=True)
    parser.add_argument("--base-url", default=None, help="OpenAI-compatible API base URL, such as ***)
    parser.add_argument("--api-key-env", default="OPENAI_API_KEY", help="Environment variable containing the
provider API key")
    parser.add_argument("--jobs", type=Path, default=Path("data/processed/preannotation/jobs.jsonl"))
    parser.add_argument("--schema", type=Path, default=Path("schemas/preannotation_candidate.schema.json"))
    parser.add_argument("--output", type=Path, default=Path("data/processed/preannotation/candidates.jsonl"))
    parser.add_argument("--log", type=Path, default=Path("outputs/preannotation_run.jsonl"))
    parser.add_argument("--max-jobs", type=int, default=1)
    parser.add_argument("--job-id")
    parser.add_argument("--timeout", type=int, default=600)
    parser.add_argument("--retries", type=int, default=3, help="Retries for transient provider errors")
    parser.add_argument("--repair-schema", action="store_true", help="Append strict field and enum repair
constraints")
    parser.add_argument("--resume", action="store_true")
    parser.add_argument("--think", action="store_true")
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(run(parse_args()))

```

scripts/run_qlora_inference.py

```

#!/usr/bin/env python3
"""Run local QLoRA adapter inference for experiment jobs."""

from __future__ import annotations

import argparse
import json

```

```

import re
import time
from pathlib import Path
from typing import Any

def load_jsonl(path: Path) -> List[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def parse_json(text: str) -> dict[str, Any]:
    starts = [match.start() for match in re.finditer(r"\{", text)]
    for start in starts:
        try:
            value, _ = json.JSONDecoder().raw_decode(text[start:])
            if isinstance(value, dict):
                return value
        except json.JSONDecodeError:
            continue
    raise ValueError("model output did not contain a complete JSON object")

def payload(job: dict[str, Any]) -> str:
    return json.dumps(
        {
            "document_id": job["document_id"],
            "language": job["language"],
            "teacher_model": job.get("teacher_model", "qwen3-4b-qlora"),
            "ontology": job["ontology"],
            "segments": job["segments"],
        },
        ensure_ascii=False,
    )

def main(args: argparse.Namespace) -> int:
    import torch
    from peft import PeftModel
    from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

    tokenizer = AutoTokenizer.from_pretrained(args.adapter, local_files_only=True, trust_remote_code=True)
    quantization = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_use_double_quant=True,
    )
    base = AutoModelForCausalLM.from_pretrained(
        args.model_path,
        local_files_only=True,
        trust_remote_code=True,
        quantization_config=quantization,
        device_map={"": 0},
        torch_dtype=torch.bfloat16,
    )
    model = PeftModel.from_pretrained(base, args.adapter, local_files_only=True)
    model.eval()
    device = next(model.parameters()).device
    args.output.parent.mkdir(parents=True, exist_ok=True)
    with args.output.open("w", encoding="utf-8") as output, args.log.open("w", encoding="utf-8") as log:
        jobs = load_jsonl(args.jobs)
        for index, job in enumerate(jobs, 1):
            started = time.monotonic()
            messages = [
                {"role": "system", "content": job["system_instruction"]},
                {"role": "user", "content": payload(job)},
            ]
            prompt = tokenizer.apply_chat_template(
                messages,
                tokenize=False,
                add_generation_prompt=True,

```



```

        enable_thinking=False,
    )
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True,
max_length=args.max_input_tokens).to(device)
    with torch.inference_mode():
        generated = model.generate(
            **inputs,
            max_new_tokens=args.max_new_tokens,
            do_sample=False,
            temperature=None,
            top_p=None,
            pad_token_id=tokenizer.eos_token_id,
        )
    text = tokenizer.decode(generated[0][inputs["input_ids"].shape[1] :], skip_special_tokens=True)
    try:
        annotation = parse_json(text)
    except Exception as error:
        log.write(json.dumps({"job_id": job["job_id"], "status": "failed", "error": str(error), "raw_tail":
text[-500:]}), ensure_ascii=False) + "\n")
        log.flush()
        print(f"{index}/{len(jobs)} {job['job_id']} failed: {error}", flush=True)
        continue
    annotation.setdefault("schema_version", "0.1.0")
    annotation.setdefault("document_id", job["document_id"])
    annotation.setdefault("language", job["language"])
    annotation.setdefault("review", {"status": "unreviewed", "reviewers": [], "notes": "qlora inference"})
    for entity in annotation.get("entities", []):
        entity.setdefault("created_by", "qwen3-4b-qlora")
        entity.setdefault("review_status", "pending")
    for relation in annotation.get("relations", []):
        relation.setdefault("created_by", "qwen3-4b-qlora")
        relation.setdefault("review_status", "pending")
    output.write(json.dumps({"job_id": job["job_id"], "annotation": annotation}, ensure_ascii=False) +
"\n")
    output.flush()
    log.write(json.dumps({"job_id": job["job_id"], "status": "success", "elapsed_seconds":
round(time.monotonic() - started, 3)}, ensure_ascii=False) + "\n")
    log.flush()
    print(f"{index}/{len(jobs)} {job['job_id']} success", flush=True)
    return 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--model-path", type=Path, required=True)
    parser.add_argument("--adapter", type=Path, required=True)
    parser.add_argument("--jobs", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--log", type=Path, required=True)
    parser.add_argument("--max-input-tokens", type=int, default=12288)
    parser.add_argument("--max-new-tokens", type=int, default=4096)
    return parser.parse_args()

if __name__ == "__main__":
    raise SystemExit(main(parse_args()))

```

scripts/select_pilot_set.py

```

#!/usr/bin/env python3
"""Select a deterministic, diverse pilot set from successfully extracted documents."""

from __future__ import annotations

import argparse
import csv
import json
import re
from collections import defaultdict

```

```

from pathlib import Path
from typing import Any

ENGLISH_CATEGORIES = [
    ("collision", re.compile(r"\bcollision|collided|struck\b", re.IGNORECASE)),
    ("derailment", re.compile(r"\bderail(?:ment|ed)?\b", re.IGNORECASE)),
    ("level_crossing", re.compile(r"\blevel crossing\b", re.IGNORECASE)),
    ("fire", re.compile(r"\bfire|burning|smoke\b", re.IGNORECASE)),
    ("signal_or_spad", re.compile(r"\bsignal|SPAD|danger aspect\b", re.IGNORECASE)),
    ("infrastructure", re.compile(r"\btrack|switch|points|bridge|landslip\b", re.IGNORECASE)),
]

CHINESE_CATEGORIES = [
    ("railway", re.compile(r"铁路|铁道|地铁|列车|轨道|道岔|信号")),
    ("fire", re.compile(r"消防|火灾|着火|自燃")),
    ("hazardous_material", re.compile(r"危化|危险化学品|危险品|燃气|泄漏")),
    ("construction", re.compile(r"施工|建筑|工程")),
    ("emergency_drill", re.compile(r"应急演练|演练方案")),
    ("management", re.compile(r"管理制度|责任制|检查表|操作规程")),
]

def classify(text: str, patterns: list[tuple[str, re.Pattern[str]]]) -> str:
    for category, pattern in patterns:
        if pattern.search(text):
            return category
    return "other"

def load_document(document_root: Path, document_id: str) -> dict[str, Any]:
    path = document_root / f"{document_id}.json"
    return json.loads(path.read_text(encoding="utf-8"))

def select(args: argparse.Namespace) -> None:
    with args.manifest.open(encoding="utf-8-sig", newline="") as stream:
        rows = list(csv.DictReader(stream))

    candidates: dict[tuple[str, str], list[dict[str, str]]] = defaultdict(list)
    for row in rows:
        if row["extract_status"] != "success" or row["duplicate_of"]:
            continue
        document = load_document(args.document_root, row["document_id"])
        if row["source_group"] == "raib":
            category = classify(document["text"][:50000], ENGLISH_CATEGORIES)
            language = "en"
        elif row["source_group"] == "cn":
            path_parts = Path(row["relative_path"]).parts
            # The downloaded bundle root contains generic words such as "消防" in
            # every path, so only classify on inner directories and file content.
            inner_path = "/".join(path_parts[2:])
            category = classify(inner_path, CHINESE_CATEGORIES)
            language = "zh"
        else:
            continue
        candidate = dict(row)
        candidate["category"] = category
        candidate["pilot_language"] = language
        candidates[(language, category)].append(candidate)

    selected: list[dict[str, str]] = []
    for key in sorted(candidates):
        group = sorted(candidates[key], key=lambda item: (int(item["char_count"]), item["document_id"]))
        # Avoid tiny files and favor documents near the group median rather than the largest reports.
        substantive = [item for item in group if int(item["char_count"]) >= args.min_characters]
        if not substantive:
            substantive = group
        if not substantive:
            continue
        positions = []

```

```

    if args.per_category == 1:
        positions = [len(substantive) // 2]
    else:
        positions = [
            round((index + 1) * (len(substantive) - 1) / (args.per_category + 1))
            for index in range(args.per_category)
        ]
    seen: set[str] = set()
    for position in positions:
        item = substantive[position]
        if item["document_id"] in seen:
            continue
        seen.add(item["document_id"])
        selected.append(item)

args.output.parent.mkdir(parents=True, exist_ok=True)
fieldnames = [
    "document_id",
    "pilot_language",
    "source_group",
    "category",
    "relative_path",
    "actual_format",
    "char_count",
    "sha256",
    "review_status",
]
with args.output.open("w", encoding="utf-8-sig", newline="") as stream:
    writer = csv.DictWriter(stream, fieldnames=fieldnames)
    writer.writeheader()
    for item in selected:
        writer.writerow({**{field: item.get(field, "") for field in fieldnames}, "review_status": "pending"})
counts: dict[str, int] = defaultdict(int)
for item in selected:
    counts[f"{item['pilot_language']}:{item['category']}"] += 1
print(json.dumps({"selected": len(selected), "groups": counts}, ensure_ascii=False, indent=2))

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--manifest", type=Path, default=Path("data/catalog/corpus_inventory.csv"))
    parser.add_argument("--document-root", type=Path, default=Path("data/processed/corpus/documents"))
    parser.add_argument("--output", type=Path, default=Path("data/catalog/pilot_set.csv"))
    parser.add_argument("--per-category", type=int, default=2)
    parser.add_argument("--min-characters", type=int, default=2000)
    return parser.parse_args()

if __name__ == "__main__":
    select(parse_args())

```

scripts/serve_review.py

```

#!/usr/bin/env python3
"""Serve the browser-based annotation review workbench on the local network."""

from __future__ import annotations

import argparse
import functools
import http.server
import socket
from pathlib import Path

def local_ip() -> str:
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        sock.connect(("***", 80))

```

```

        return sock.getsockname()[0]
    except OSError:
        return ""
    finally:
        sock.close()

def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--host", default="")
    parser.add_argument("--port", type=int, default=8765)
    args = parser.parse_args()
    root = Path(__file__).resolve().parents[1]
    handler = functools.partial(http.server.SimpleHTTPRequestHandler, directory=str(root))
    server = http.server.ThreadingHTTPServer((args.host, args.port), handler)
    print(f"Review page: ***")
    print(f"LAN address: ***}:{args.port}/web/review.html")
    print("Press Ctrl-C to stop.")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("Stopping review server.")
    finally:
        server.server_close()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

scripts/train_qlora.py

```

#!/usr/bin/env python3
"""Train a small QLoRA adapter for evidence-grounded safety annotation."""

from __future__ import annotations

import argparse
import json
import math
from pathlib import Path
from typing import Any

def load_jsonl(path: Path) -> list[dict[str, Any]]:
    return [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]

def build_examples(gold: Path, index: Path, jobs: Path) -> list[dict[str, str]]:
    annotations = load_jsonl(gold)
    indexes = load_jsonl(index)
    jobs_by_id = {row["job_id"]: row for row in load_jsonl(jobs)}
    if len(annotations) != len(indexes):
        raise ValueError("gold and index record counts differ")
    examples = []
    for annotation, record in zip(annotations, indexes):
        job = jobs_by_id[record["job_id"]]
        user = json.dumps(
            {
                "document_id": job["document_id"],
                "language": job["language"],
                "teacher_model": "qlora-student",
                "ontology": job["ontology"],
                "segments": job["segments"],
            },
            ensure_ascii=False,
        )
        target = json.dumps(annotation, ensure_ascii=False)
        examples.append({"system": job["system_instruction"], "user": user, "target": target})

```

return examples

```
def main(args: argparse.Namespace) -> int:
    import torch
    from accelerate import Accelerator
    from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
    from torch.utils.data import DataLoader
    from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

    examples = build_examples(args.gold, args.index, args.jobs)
    tokenizer = AutoTokenizer.from_pretrained(args.model_path, local_files_only=True, trust_remote_code=True)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    quantization = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_use_double_quant=True,
    )
    model = AutoModelForCausalLM.from_pretrained(
        args.model_path,
        local_files_only=True,
        trust_remote_code=True,
        quantization_config=quantization,
        device_map={"": 0},
        torch_dtype=torch.bfloat16,
    )
    model = prepare_model_for_kbit_training(model)
    model.gradient_checkpointing_enable()
    model = get_peft_model(
        model,
        LoraConfig(
            r=args.lora_rank,
            lora_alpha=args.lora_alpha,
            lora_dropout=0.05,
            bias="none",
            task_type="CAUSAL_LM",
            target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "up_proj", "down_proj", "gate_proj"],
        ),
    )
    model.print_trainable_parameters()

    device = torch.device("cuda:0")
    encoded = []
    for example in examples:
        prompt = (
            "<|im_start|>system\n"
            + example["system"]
            + "<|im_end|>\n<|im_start|>user\n"
            + example["user"]
            + "<|im_end|>\n<|im_start|>assistant\n"
        )
        full = prompt + example["target"] + "<|im_end|>"
        prompt_ids = tokenizer(prompt, add_special_tokens=False)["input_ids"]
        answer_ids = tokenizer(example["target"] + "<|im_end|>", add_special_tokens=False)["input_ids"]
        if len(answer_ids) >= args.max_length:
            answer_ids = answer_ids[: args.max_length - 1]
        prompt_budget = args.max_length - len(answer_ids)
        if prompt_budget < 1:
            continue
        # Keep the beginning of the prompt and the complete available answer so
        # truncation never creates a batch with zero supervised tokens.
        prompt_ids = prompt_ids[:prompt_budget]
        full_ids = prompt_ids + answer_ids
        labels = [-100] * len(prompt_ids) + answer_ids
        encoded.append({"input_ids": full_ids, "labels": labels})

    if not encoded:
        raise ValueError("no train examples contain supervised answer tokens")
    print(json.dumps({"prepared_examples": len(encoded), "max_sequence": max(len(row["input_ids"]) for row in
```

```

encoded)), ensure_ascii=False), flush=True)

def collate(batch: list[dict[str, list[int]]]) -> dict[str, torch.Tensor]:
    max_len = max(len(row["input_ids"]) for row in batch)
    input_ids = [row["input_ids"] + [tokenizer.pad_token_id] * (max_len - len(row["input_ids"])) for row in
batch]
    labels = [row["labels"] + [-100] * (max_len - len(row["labels"])) for row in batch]
    attention = [[1] * len(row["input_ids"]) + [0] * (max_len - len(row["input_ids"])) for row in batch]
    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long),
        "attention_mask": torch.tensor(attention, dtype=torch.long),
    }

loader = DataLoader(encoded, batch_size=args.batch_size, shuffle=True, collate_fn=collate)
optimizer = torch.optim.AdamW((p for p in model.parameters() if p.requires_grad), lr=args.learning_rate)
accelerator = Accelerator(gradient_accumulation_steps=args.gradient_accumulation)
model, optimizer, loader = accelerator.prepare(model, optimizer, loader)
model.train()
steps = 0
losses = []
for epoch in range(args.epochs):
    for batch in loader:
        with accelerator.accumulate(model):
            outputs = model(
                input_ids=batch["input_ids"].to(device),
                attention_mask=batch["attention_mask"].to(device),
                labels=batch["labels"].to(device),
            )
            loss = outputs.loss
            if not torch.isfinite(loss):
                raise FloatingPointError("non-finite training loss; check truncation and labels")
            accelerator.backward(loss)
            optimizer.step()
            optimizer.zero_grad()
        if accelerator.sync_gradients:
            steps += 1
            losses.append(float(loss.detach().cpu()))
            if steps % 5 == 0:
                print(json.dumps({"epoch": epoch + 1, "step": steps, "loss": round(losses[-1], 5)},
ensure_ascii=False), flush=True)

args.output.mkdir(parents=True, exist_ok=True)
accelerator.unwrap_model(model).save_pretrained(args.output)
tokenizer.save_pretrained(args.output)
metrics = {
    "base_model": str(args.model_path),
    "train_examples": len(examples),
    "epochs": args.epochs,
    "steps": steps,
    "final_loss": losses[-1] if losses else None,
    "mean_loss": sum(losses) / len(losses) if losses else None,
    "perplexity_estimate": math.exp(min(sum(losses) / len(losses), 20)) if losses else None,
    "max_length": args.max_length,
    "lora_rank": args.lora_rank,
}
(args.output / "training_metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent=2) + "\n",
encoding="utf-8")
print(json.dumps(metrics, ensure_ascii=False, indent=2))
return 0

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--model-path", type=Path, required=True)
    parser.add_argument("--gold", type=Path, default=Path("data/processed/reviewed/gold/train.jsonl"))
    parser.add_argument("--index", type=Path, default=Path("data/processed/reviewed/gold/train_index.jsonl"))
    parser.add_argument("--jobs", type=Path, default=Path("data/processed/experiments/train_kg_jobs.jsonl"))
    parser.add_argument("--output", type=Path, default=Path("data/processed/experiments/qwen3_4b_kg_qlora"))
    parser.add_argument("--epochs", type=int, default=1)
    parser.add_argument("--batch-size", type=int, default=1)

```

0x87	0x2021	#DOUBLE DAGGER
0x88	0x20AC	#EURO SIGN
0x89	0x2030	#PER MILLE SIGN
0x8A	0x0409	#CYRILLIC CAPITAL LETTER LJE
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C	0x040A	#CYRILLIC CAPITAL LETTER NJE
0x8D	0x040C	#CYRILLIC CAPITAL LETTER KJE
0x8E	0x040B	#CYRILLIC CAPITAL LETTER TSHE
0x8F	0x040F	#CYRILLIC CAPITAL LETTER DZHE
0x90	0x0452	#CYRILLIC SMALL LETTER DJE
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98		#UNDEFINED
0x99	0x2122	#TRADE MARK SIGN
0x9A	0x0459	#CYRILLIC SMALL LETTER LJE
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C	0x045A	#CYRILLIC SMALL LETTER NJE
0x9D	0x045C	#CYRILLIC SMALL LETTER KJE
0x9E	0x045B	#CYRILLIC SMALL LETTER TSHE
0x9F	0x045F	#CYRILLIC SMALL LETTER DZHE
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x040E	#CYRILLIC CAPITAL LETTER SHORT U
0xA2	0x045E	#CYRILLIC SMALL LETTER SHORT U
0xA3	0x0408	#CYRILLIC CAPITAL LETTER JE
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x0490	#CYRILLIC CAPITAL LETTER GHE WITH UPTURN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x0401	#CYRILLIC CAPITAL LETTER IO
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x0404	#CYRILLIC CAPITAL LETTER UKRAINIAN IE
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x0407	#CYRILLIC CAPITAL LETTER YI
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x0406	#CYRILLIC CAPITAL LETTER BYELORUSSIAN-UKRAINIAN I
0xB3	0x0456	#CYRILLIC SMALL LETTER BYELORUSSIAN-UKRAINIAN I
0xB4	0x0491	#CYRILLIC SMALL LETTER GHE WITH UPTURN
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x0451	#CYRILLIC SMALL LETTER IO
0xB9	0x2116	#NUMERO SIGN
0xBA	0x0454	#CYRILLIC SMALL LETTER UKRAINIAN IE
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x0458	#CYRILLIC SMALL LETTER JE
0xBD	0x0405	#CYRILLIC CAPITAL LETTER DZE
0xBE	0x0455	#CYRILLIC SMALL LETTER DZE
0xBF	0x0457	#CYRILLIC SMALL LETTER YI
0xC0	0x0410	#CYRILLIC CAPITAL LETTER A
0xC1	0x0411	#CYRILLIC CAPITAL LETTER BE
0xC2	0x0412	#CYRILLIC CAPITAL LETTER VE
0xC3	0x0413	#CYRILLIC CAPITAL LETTER GHE
0xC4	0x0414	#CYRILLIC CAPITAL LETTER DE
0xC5	0x0415	#CYRILLIC CAPITAL LETTER IE
0xC6	0x0416	#CYRILLIC CAPITAL LETTER ZHE
0xC7	0x0417	#CYRILLIC CAPITAL LETTER ZE
0xC8	0x0418	#CYRILLIC CAPITAL LETTER I
0xC9	0x0419	#CYRILLIC CAPITAL LETTER SHORT I
0xCA	0x041A	#CYRILLIC CAPITAL LETTER KA
0xCB	0x041B	#CYRILLIC CAPITAL LETTER EL
0xCC	0x041C	#CYRILLIC CAPITAL LETTER EM
0xCD	0x041D	#CYRILLIC CAPITAL LETTER EN

0xCE	0x041E	#CYRILLIC CAPITAL LETTER O
0xCF	0x041F	#CYRILLIC CAPITAL LETTER PE
0xD0	0x0420	#CYRILLIC CAPITAL LETTER ER
0xD1	0x0421	#CYRILLIC CAPITAL LETTER ES
0xD2	0x0422	#CYRILLIC CAPITAL LETTER TE
0xD3	0x0423	#CYRILLIC CAPITAL LETTER U
0xD4	0x0424	#CYRILLIC CAPITAL LETTER EF
0xD5	0x0425	#CYRILLIC CAPITAL LETTER HA
0xD6	0x0426	#CYRILLIC CAPITAL LETTER TSE
0xD7	0x0427	#CYRILLIC CAPITAL LETTER CHE
0xD8	0x0428	#CYRILLIC CAPITAL LETTER SHA
0xD9	0x0429	#CYRILLIC CAPITAL LETTER SHCHA
0xDA	0x042A	#CYRILLIC CAPITAL LETTER HARD SIGN
0xDB	0x042B	#CYRILLIC CAPITAL LETTER YERU
0xDC	0x042C	#CYRILLIC CAPITAL LETTER SOFT SIGN
0xDD	0x042D	#CYRILLIC CAPITAL LETTER E
0xDE	0x042E	#CYRILLIC CAPITAL LETTER YU
0xDF	0x042F	#CYRILLIC CAPITAL LETTER YA
0xE0	0x0430	#CYRILLIC SMALL LETTER A
0xE1	0x0431	#CYRILLIC SMALL LETTER BE
0xE2	0x0432	#CYRILLIC SMALL LETTER VE
0xE3	0x0433	#CYRILLIC SMALL LETTER GHE
0xE4	0x0434	#CYRILLIC SMALL LETTER DE
0xE5	0x0435	#CYRILLIC SMALL LETTER IE
0xE6	0x0436	#CYRILLIC SMALL LETTER ZHE
0xE7	0x0437	#CYRILLIC SMALL LETTER ZE
0xE8	0x0438	#CYRILLIC SMALL LETTER I
0xE9	0x0439	#CYRILLIC SMALL LETTER SHORT I
0xEA	0x043A	#CYRILLIC SMALL LETTER KA
0xEB	0x043B	#CYRILLIC SMALL LETTER EL
0xEC	0x043C	#CYRILLIC SMALL LETTER EM
0xED	0x043D	#CYRILLIC SMALL LETTER EN
0xEE	0x043E	#CYRILLIC SMALL LETTER O
0xEF	0x043F	#CYRILLIC SMALL LETTER PE
0xF0	0x0440	#CYRILLIC SMALL LETTER ER
0xF1	0x0441	#CYRILLIC SMALL LETTER ES
0xF2	0x0442	#CYRILLIC SMALL LETTER TE
0xF3	0x0443	#CYRILLIC SMALL LETTER U
0xF4	0x0444	#CYRILLIC SMALL LETTER EF
0xF5	0x0445	#CYRILLIC SMALL LETTER HA
0xF6	0x0446	#CYRILLIC SMALL LETTER TSE
0xF7	0x0447	#CYRILLIC SMALL LETTER CHE
0xF8	0x0448	#CYRILLIC SMALL LETTER SHA
0xF9	0x0449	#CYRILLIC SMALL LETTER SHCHA
0xFA	0x044A	#CYRILLIC SMALL LETTER HARD SIGN
0xFB	0x044B	#CYRILLIC SMALL LETTER YERU
0xFC	0x044C	#CYRILLIC SMALL LETTER SOFT SIGN
0xFD	0x044D	#CYRILLIC SMALL LETTER E
0xFE	0x044E	#CYRILLIC SMALL LETTER YU
0xFF	0x044F	#CYRILLIC SMALL LETTER YA

tools/catdoc/root/usr/share/catdoc/cp1252.txt

```
#
# Name:      cp1252 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      04/15/98
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1252 code (in hex)
#      Column #2 is the Unicode (in hex as 0XXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
```


The entries are in cp1252 order

```
#
0x00 0x0000 #NULL
0x01 0x0001 #START OF HEADING
0x02 0x0002 #START OF TEXT
0x03 0x0003 #END OF TEXT
0x04 0x0004 #END OF TRANSMISSION
0x05 0x0005 #ENQUIRY
0x06 0x0006 #ACKNOWLEDGE
0x07 0x0007 #BELL
0x08 0x0008 #BACKSPACE
0x09 0x0009 #HORIZONTAL TABULATION
0x0A 0x000A #LINE FEED
0x0B 0x000B #VERTICAL TABULATION
0x0C 0x000C #FORM FEED
0x0D 0x000D #CARRIAGE RETURN
0x0E 0x000E #SHIFT OUT
0x0F 0x000F #SHIFT IN
0x10 0x0010 #DATA LINK ESCAPE
0x11 0x0011 #DEVICE CONTROL ONE
0x12 0x0012 #DEVICE CONTROL TWO
0x13 0x0013 #DEVICE CONTROL THREE
0x14 0x0014 #DEVICE CONTROL FOUR
0x15 0x0015 #NEGATIVE ACKNOWLEDGE
0x16 0x0016 #SYNCHRONOUS IDLE
0x17 0x0017 #END OF TRANSMISSION BLOCK
0x18 0x0018 #CANCEL
0x19 0x0019 #END OF MEDIUM
0x1A 0x001A #SUBSTITUTE
0x1B 0x001B #ESCAPE
0x1C 0x001C #FILE SEPARATOR
0x1D 0x001D #GROUP SEPARATOR
0x1E 0x001E #RECORD SEPARATOR
0x1F 0x001F #UNIT SEPARATOR
0x20 0x0020 #SPACE
0x21 0x0021 #EXCLAMATION MARK
0x22 0x0022 #QUOTATION MARK
0x23 0x0023 #NUMBER SIGN
0x24 0x0024 #DOLLAR SIGN
0x25 0x0025 #PERCENT SIGN
0x26 0x0026 #AMPERSAND
0x27 0x0027 #APOSTROPHE
0x28 0x0028 #LEFT PARENTHESIS
0x29 0x0029 #RIGHT PARENTHESIS
0x2A 0x002A #ASTERISK
0x2B 0x002B #PLUS SIGN
0x2C 0x002C #COMMA
0x2D 0x002D #HYPHEN-MINUS
0x2E 0x002E #FULL STOP
0x2F 0x002F #SOLIDUS
0x30 0x0030 #DIGIT ZERO
0x31 0x0031 #DIGIT ONE
0x32 0x0032 #DIGIT TWO
0x33 0x0033 #DIGIT THREE
0x34 0x0034 #DIGIT FOUR
0x35 0x0035 #DIGIT FIVE
0x36 0x0036 #DIGIT SIX
0x37 0x0037 #DIGIT SEVEN
0x38 0x0038 #DIGIT EIGHT
0x39 0x0039 #DIGIT NINE
0x3A 0x003A #COLON
0x3B 0x003B #SEMICOLON
0x3C 0x003C #LESS-THAN SIGN
0x3D 0x003D #EQUALS SIGN
0x3E 0x003E #GREATER-THAN SIGN
0x3F 0x003F #QUESTION MARK
0x40 0x0040 #COMMERCIAL AT
0x41 0x0041 #LATIN CAPITAL LETTER A
0x42 0x0042 #LATIN CAPITAL LETTER B
0x43 0x0043 #LATIN CAPITAL LETTER C
0x44 0x0044 #LATIN CAPITAL LETTER D
```

0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88	0x02C6	#MODIFIER LETTER CIRCUMFLEX ACCENT
0x89	0x2030	#PER MILLE SIGN
0x8A	0x0160	#LATIN CAPITAL LETTER S WITH CARON
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK

0x8C	0x0152	#LATIN CAPITAL LIGATURE OE
0x8D		#UNDEFINED
0x8E	0x017D	#LATIN CAPITAL LETTER Z WITH CARON
0x8F		#UNDEFINED
0x90		#UNDEFINED
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98	0x02DC	#SMALL TILDE
0x99	0x2122	#TRADE MARK SIGN
0x9A	0x0161	#LATIN SMALL LETTER S WITH CARON
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C	0x0153	#LATIN SMALL LIGATURE OE
0x9D		#UNDEFINED
0x9E	0x017E	#LATIN SMALL LETTER Z WITH CARON
0x9F	0x0178	#LATIN CAPITAL LETTER Y WITH DIAERESIS
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x00A1	#INVERTED EXCLAMATION MARK
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x00AA	#FEMININE ORDINAL INDICATOR
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00AF	#MACRON
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERScript TWO
0xB3	0x00B3	#SUPERScript THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00B8	#CEDILLA
0xB9	0x00B9	#SUPERScript ONE
0xBA	0x00BA	#MASCULINE ORDINAL INDICATOR
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x00BF	#INVERTED QUESTION MARK
0xC0	0x00C0	#LATIN CAPITAL LETTER A WITH GRAVE
0xC1	0x00C1	#LATIN CAPITAL LETTER A WITH ACUTE
0xC2	0x00C2	#LATIN CAPITAL LETTER A WITH CIRCUMFLEX
0xC3	0x00C3	#LATIN CAPITAL LETTER A WITH TILDE
0xC4	0x00C4	#LATIN CAPITAL LETTER A WITH DIAERESIS
0xC5	0x00C5	#LATIN CAPITAL LETTER A WITH RING ABOVE
0xC6	0x00C6	#LATIN CAPITAL LETTER AE
0xC7	0x00C7	#LATIN CAPITAL LETTER C WITH CEDILLA
0xC8	0x00C8	#LATIN CAPITAL LETTER E WITH GRAVE
0xC9	0x00C9	#LATIN CAPITAL LETTER E WITH ACUTE
0xCA	0x00CA	#LATIN CAPITAL LETTER E WITH CIRCUMFLEX
0xCB	0x00CB	#LATIN CAPITAL LETTER E WITH DIAERESIS
0xCC	0x00CC	#LATIN CAPITAL LETTER I WITH GRAVE
0xCD	0x00CD	#LATIN CAPITAL LETTER I WITH ACUTE
0xCE	0x00CE	#LATIN CAPITAL LETTER I WITH CIRCUMFLEX
0xCF	0x00CF	#LATIN CAPITAL LETTER I WITH DIAERESIS
0xD0	0x00D0	#LATIN CAPITAL LETTER ETH
0xD1	0x00D1	#LATIN CAPITAL LETTER N WITH TILDE
0xD2	0x00D2	#LATIN CAPITAL LETTER O WITH GRAVE

0xD3	0x00D3	#LATIN CAPITAL LETTER O WITH ACUTE
0xD4	0x00D4	#LATIN CAPITAL LETTER O WITH CIRCUMFLEX
0xD5	0x00D5	#LATIN CAPITAL LETTER O WITH TILDE
0xD6	0x00D6	#LATIN CAPITAL LETTER O WITH DIAERESIS
0xD7	0x00D7	#MULTIPLICATION SIGN
0xD8	0x00D8	#LATIN CAPITAL LETTER O WITH STROKE
0xD9	0x00D9	#LATIN CAPITAL LETTER U WITH GRAVE
0xDA	0x00DA	#LATIN CAPITAL LETTER U WITH ACUTE
0xDB	0x00DB	#LATIN CAPITAL LETTER U WITH CIRCUMFLEX
0xDC	0x00DC	#LATIN CAPITAL LETTER U WITH DIAERESIS
0xDD	0x00DD	#LATIN CAPITAL LETTER Y WITH ACUTE
0xDE	0x00DE	#LATIN CAPITAL LETTER THORN
0xDF	0x00DF	#LATIN SMALL LETTER SHARP S
0xE0	0x00E0	#LATIN SMALL LETTER A WITH GRAVE
0xE1	0x00E1	#LATIN SMALL LETTER A WITH ACUTE
0xE2	0x00E2	#LATIN SMALL LETTER A WITH CIRCUMFLEX
0xE3	0x00E3	#LATIN SMALL LETTER A WITH TILDE
0xE4	0x00E4	#LATIN SMALL LETTER A WITH DIAERESIS
0xE5	0x00E5	#LATIN SMALL LETTER A WITH RING ABOVE
0xE6	0x00E6	#LATIN SMALL LETTER AE
0xE7	0x00E7	#LATIN SMALL LETTER C WITH CEDILLA
0xE8	0x00E8	#LATIN SMALL LETTER E WITH GRAVE
0xE9	0x00E9	#LATIN SMALL LETTER E WITH ACUTE
0xEA	0x00EA	#LATIN SMALL LETTER E WITH CIRCUMFLEX
0xEB	0x00EB	#LATIN SMALL LETTER E WITH DIAERESIS
0xEC	0x00EC	#LATIN SMALL LETTER I WITH GRAVE
0xED	0x00ED	#LATIN SMALL LETTER I WITH ACUTE
0xEE	0x00EE	#LATIN SMALL LETTER I WITH CIRCUMFLEX
0xEF	0x00EF	#LATIN SMALL LETTER I WITH DIAERESIS
0xF0	0x00F0	#LATIN SMALL LETTER ETH
0xF1	0x00F1	#LATIN SMALL LETTER N WITH TILDE
0xF2	0x00F2	#LATIN SMALL LETTER O WITH GRAVE
0xF3	0x00F3	#LATIN SMALL LETTER O WITH ACUTE
0xF4	0x00F4	#LATIN SMALL LETTER O WITH CIRCUMFLEX
0xF5	0x00F5	#LATIN SMALL LETTER O WITH TILDE
0xF6	0x00F6	#LATIN SMALL LETTER O WITH DIAERESIS
0xF7	0x00F7	#DIVISION SIGN
0xF8	0x00F8	#LATIN SMALL LETTER O WITH STROKE
0xF9	0x00F9	#LATIN SMALL LETTER U WITH GRAVE
0xFA	0x00FA	#LATIN SMALL LETTER U WITH ACUTE
0xFB	0x00FB	#LATIN SMALL LETTER U WITH CIRCUMFLEX
0xFC	0x00FC	#LATIN SMALL LETTER U WITH DIAERESIS
0xFD	0x00FD	#LATIN SMALL LETTER Y WITH ACUTE
0xFE	0x00FE	#LATIN SMALL LETTER THORN
0xFF	0x00FF	#LATIN SMALL LETTER Y WITH DIAERESIS

tools/catdoc/root/usr/share/catdoc/cp1253.txt

```
#
# Name:      cp1253 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      04/15/98
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1253 code (in hex)
#      Column #2 is the Unicode (in hex as 0xXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1253 order
#
0x00      0x0000      #NULL
0x01      0x0001      #START OF HEADING
0x02      0x0002      #START OF TEXT
```

0x03	0x0003	#END OF TEXT
0x04	0x0004	#END OF TRANSMISSION
0x05	0x0005	#ENQUIRY
0x06	0x0006	#ACKNOWLEDGE
0x07	0x0007	#BELL
0x08	0x0008	#BACKSPACE
0x09	0x0009	#HORIZONTAL TABULATION
0x0A	0x000A	#LINE FEED
0x0B	0x000B	#VERTICAL TABULATION
0x0C	0x000C	#FORM FEED
0x0D	0x000D	#CARRIAGE RETURN
0x0E	0x000E	#SHIFT OUT
0x0F	0x000F	#SHIFT IN
0x10	0x0010	#DATA LINK ESCAPE
0x11	0x0011	#DEVICE CONTROL ONE
0x12	0x0012	#DEVICE CONTROL TWO
0x13	0x0013	#DEVICE CONTROL THREE
0x14	0x0014	#DEVICE CONTROL FOUR
0x15	0x0015	#NEGATIVE ACKNOWLEDGE
0x16	0x0016	#SYNCHRONOUS IDLE
0x17	0x0017	#END OF TRANSMISSION BLOCK
0x18	0x0018	#CANCEL
0x19	0x0019	#END OF MEDIUM
0x1A	0x001A	#SUBSTITUTE
0x1B	0x001B	#ESCAPE
0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I

0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88		#UNDEFINED
0x89	0x2030	#PER MILLE SIGN
0x8A		#UNDEFINED
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C		#UNDEFINED
0x8D		#UNDEFINED
0x8E		#UNDEFINED
0x8F		#UNDEFINED
0x90		#UNDEFINED

0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98		#UNDEFINED
0x99	0x2122	#TRADE MARK SIGN
0x9A		#UNDEFINED
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C		#UNDEFINED
0x9D		#UNDEFINED
0x9E		#UNDEFINED
0x9F		#UNDEFINED
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x0385	#GREEK DIALYTIKA TONOS
0xA2	0x0386	#GREEK CAPITAL LETTER ALPHA WITH TONOS
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA		#UNDEFINED
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x2015	#HORIZONTAL BAR
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERSCRIFT TWO
0xB3	0x00B3	#SUPERSCRIFT THREE
0xB4	0x0384	#GREEK TONOS
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x0388	#GREEK CAPITAL LETTER EPSILON WITH TONOS
0xB9	0x0389	#GREEK CAPITAL LETTER ETA WITH TONOS
0xBA	0x038A	#GREEK CAPITAL LETTER IOTA WITH TONOS
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x038C	#GREEK CAPITAL LETTER OMICRON WITH TONOS
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x038E	#GREEK CAPITAL LETTER UPSILON WITH TONOS
0xBF	0x038F	#GREEK CAPITAL LETTER OMEGA WITH TONOS
0xC0	0x0390	#GREEK SMALL LETTER IOTA WITH DIALYTIKA AND TONOS
0xC1	0x0391	#GREEK CAPITAL LETTER ALPHA
0xC2	0x0392	#GREEK CAPITAL LETTER BETA
0xC3	0x0393	#GREEK CAPITAL LETTER GAMMA
0xC4	0x0394	#GREEK CAPITAL LETTER DELTA
0xC5	0x0395	#GREEK CAPITAL LETTER EPSILON
0xC6	0x0396	#GREEK CAPITAL LETTER ZETA
0xC7	0x0397	#GREEK CAPITAL LETTER ETA
0xC8	0x0398	#GREEK CAPITAL LETTER THETA
0xC9	0x0399	#GREEK CAPITAL LETTER IOTA
0xCA	0x039A	#GREEK CAPITAL LETTER KAPPA
0xCB	0x039B	#GREEK CAPITAL LETTER LAMDA
0xCC	0x039C	#GREEK CAPITAL LETTER MU
0xCD	0x039D	#GREEK CAPITAL LETTER NU
0xCE	0x039E	#GREEK CAPITAL LETTER XI
0xCF	0x039F	#GREEK CAPITAL LETTER OMICRON
0xD0	0x03A0	#GREEK CAPITAL LETTER PI
0xD1	0x03A1	#GREEK CAPITAL LETTER RHO
0xD2		#UNDEFINED
0xD3	0x03A3	#GREEK CAPITAL LETTER SIGMA
0xD4	0x03A4	#GREEK CAPITAL LETTER TAU
0xD5	0x03A5	#GREEK CAPITAL LETTER UPSILON
0xD6	0x03A6	#GREEK CAPITAL LETTER PHI
0xD7	0x03A7	#GREEK CAPITAL LETTER CHI

0xD8	0x03A8	#GREEK CAPITAL LETTER PSI
0xD9	0x03A9	#GREEK CAPITAL LETTER OMEGA
0xDA	0x03AA	#GREEK CAPITAL LETTER IOTA WITH DIALYTIKA
0xDB	0x03AB	#GREEK CAPITAL LETTER UPSILON WITH DIALYTIKA
0xDC	0x03AC	#GREEK SMALL LETTER ALPHA WITH TONOS
0xDD	0x03AD	#GREEK SMALL LETTER EPSILON WITH TONOS
0xDE	0x03AE	#GREEK SMALL LETTER ETA WITH TONOS
0xDF	0x03AF	#GREEK SMALL LETTER IOTA WITH TONOS
0xE0	0x03B0	#GREEK SMALL LETTER UPSILON WITH DIALYTIKA AND TONOS
0xE1	0x03B1	#GREEK SMALL LETTER ALPHA
0xE2	0x03B2	#GREEK SMALL LETTER BETA
0xE3	0x03B3	#GREEK SMALL LETTER GAMMA
0xE4	0x03B4	#GREEK SMALL LETTER DELTA
0xE5	0x03B5	#GREEK SMALL LETTER EPSILON
0xE6	0x03B6	#GREEK SMALL LETTER ZETA
0xE7	0x03B7	#GREEK SMALL LETTER ETA
0xE8	0x03B8	#GREEK SMALL LETTER THETA
0xE9	0x03B9	#GREEK SMALL LETTER IOTA
0xEA	0x03BA	#GREEK SMALL LETTER KAPPA
0xEB	0x03BB	#GREEK SMALL LETTER LAMDA
0xEC	0x03BC	#GREEK SMALL LETTER MU
0xED	0x03BD	#GREEK SMALL LETTER NU
0xEE	0x03BE	#GREEK SMALL LETTER XI
0xEF	0x03BF	#GREEK SMALL LETTER OMICRON
0xF0	0x03C0	#GREEK SMALL LETTER PI
0xF1	0x03C1	#GREEK SMALL LETTER RHO
0xF2	0x03C2	#GREEK SMALL LETTER FINAL SIGMA
0xF3	0x03C3	#GREEK SMALL LETTER SIGMA
0xF4	0x03C4	#GREEK SMALL LETTER TAU
0xF5	0x03C5	#GREEK SMALL LETTER UPSILON
0xF6	0x03C6	#GREEK SMALL LETTER PHI
0xF7	0x03C7	#GREEK SMALL LETTER CHI
0xF8	0x03C8	#GREEK SMALL LETTER PSI
0xF9	0x03C9	#GREEK SMALL LETTER OMEGA
0xFA	0x03CA	#GREEK SMALL LETTER IOTA WITH DIALYTIKA
0xFB	0x03CB	#GREEK SMALL LETTER UPSILON WITH DIALYTIKA
0xFC	0x03CC	#GREEK SMALL LETTER OMICRON WITH TONOS
0xFD	0x03CD	#GREEK SMALL LETTER UPSILON WITH TONOS
0xFE	0x03CE	#GREEK SMALL LETTER OMEGA WITH TONOS
0xFF		#UNDEFINED

tools/catdoc/root/usr/share/catdoc/cp1254.txt

```
#
# Name:      cp1254 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      04/15/98
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1254 code (in hex)
#      Column #2 is the Unicode (in hex as 0xXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1254 order
#
0x00      0x0000      #NULL
0x01      0x0001      #START OF HEADING
0x02      0x0002      #START OF TEXT
0x03      0x0003      #END OF TEXT
0x04      0x0004      #END OF TRANSMISSION
0x05      0x0005      #ENQUIRY
0x06      0x0006      #ACKNOWLEDGE
0x07      0x0007      #BELL
```


0x08	0x0008	#BACKSPACE
0x09	0x0009	#HORIZONTAL TABULATION
0x0A	0x000A	#LINE FEED
0x0B	0x000B	#VERTICAL TABULATION
0x0C	0x000C	#FORM FEED
0x0D	0x000D	#CARRIAGE RETURN
0x0E	0x000E	#SHIFT OUT
0x0F	0x000F	#SHIFT IN
0x10	0x0010	#DATA LINK ESCAPE
0x11	0x0011	#DEVICE CONTROL ONE
0x12	0x0012	#DEVICE CONTROL TWO
0x13	0x0013	#DEVICE CONTROL THREE
0x14	0x0014	#DEVICE CONTROL FOUR
0x15	0x0015	#NEGATIVE ACKNOWLEDGE
0x16	0x0016	#SYNCHRONOUS IDLE
0x17	0x0017	#END OF TRANSMISSION BLOCK
0x18	0x0018	#CANCEL
0x19	0x0019	#END OF MEDIUM
0x1A	0x001A	#SUBSTITUTE
0x1B	0x001B	#ESCAPE
0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N

0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88	0x02C6	#MODIFIER LETTER CIRCUMFLEX ACCENT
0x89	0x2030	#PER MILLE SIGN
0x8A	0x0160	#LATIN CAPITAL LETTER S WITH CARON
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C	0x0152	#LATIN CAPITAL LIGATURE OE
0x8D		#UNDEFINED
0x8E		#UNDEFINED
0x8F		#UNDEFINED
0x90		#UNDEFINED
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET

0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98	0x02DC	#SMALL TILDE
0x99	0x2122	#TRADE MARK SIGN
0x9A	0x0161	#LATIN SMALL LETTER S WITH CARON
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C	0x0153	#LATIN SMALL LIGATURE OE
0x9D		#UNDEFINED
0x9E		#UNDEFINED
0x9F	0x0178	#LATIN CAPITAL LETTER Y WITH DIAERESIS
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x00A1	#INVERTED EXCLAMATION MARK
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x00AA	#FEMININE ORDINAL INDICATOR
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00AF	#MACRON
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERSCRIFT TWO
0xB3	0x00B3	#SUPERSCRIFT THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00B8	#CEDILLA
0xB9	0x00B9	#SUPERSCRIFT ONE
0xBA	0x00BA	#MASCULINE ORDINAL INDICATOR
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x00BF	#INVERTED QUESTION MARK
0xC0	0x00C0	#LATIN CAPITAL LETTER A WITH GRAVE
0xC1	0x00C1	#LATIN CAPITAL LETTER A WITH ACUTE
0xC2	0x00C2	#LATIN CAPITAL LETTER A WITH CIRCUMFLEX
0xC3	0x00C3	#LATIN CAPITAL LETTER A WITH TILDE
0xC4	0x00C4	#LATIN CAPITAL LETTER A WITH DIAERESIS
0xC5	0x00C5	#LATIN CAPITAL LETTER A WITH RING ABOVE
0xC6	0x00C6	#LATIN CAPITAL LETTER AE
0xC7	0x00C7	#LATIN CAPITAL LETTER C WITH CEDILLA
0xC8	0x00C8	#LATIN CAPITAL LETTER E WITH GRAVE
0xC9	0x00C9	#LATIN CAPITAL LETTER E WITH ACUTE
0xCA	0x00CA	#LATIN CAPITAL LETTER E WITH CIRCUMFLEX
0xCB	0x00CB	#LATIN CAPITAL LETTER E WITH DIAERESIS
0xCC	0x00CC	#LATIN CAPITAL LETTER I WITH GRAVE
0xCD	0x00CD	#LATIN CAPITAL LETTER I WITH ACUTE
0xCE	0x00CE	#LATIN CAPITAL LETTER I WITH CIRCUMFLEX
0xCF	0x00CF	#LATIN CAPITAL LETTER I WITH DIAERESIS
0xD0	0x011E	#LATIN CAPITAL LETTER G WITH BREVE
0xD1	0x00D1	#LATIN CAPITAL LETTER N WITH TILDE
0xD2	0x00D2	#LATIN CAPITAL LETTER O WITH GRAVE
0xD3	0x00D3	#LATIN CAPITAL LETTER O WITH ACUTE
0xD4	0x00D4	#LATIN CAPITAL LETTER O WITH CIRCUMFLEX
0xD5	0x00D5	#LATIN CAPITAL LETTER O WITH TILDE
0xD6	0x00D6	#LATIN CAPITAL LETTER O WITH DIAERESIS
0xD7	0x00D7	#MULTIPLICATION SIGN
0xD8	0x00D8	#LATIN CAPITAL LETTER O WITH STROKE
0xD9	0x00D9	#LATIN CAPITAL LETTER U WITH GRAVE
0xDA	0x00DA	#LATIN CAPITAL LETTER U WITH ACUTE
0xDB	0x00DB	#LATIN CAPITAL LETTER U WITH CIRCUMFLEX
0xDC	0x00DC	#LATIN CAPITAL LETTER U WITH DIAERESIS

0xDD	0x0130	#LATIN CAPITAL LETTER I WITH DOT ABOVE
0xDE	0x015E	#LATIN CAPITAL LETTER S WITH CEDILLA
0xDF	0x00DF	#LATIN SMALL LETTER SHARP S
0xE0	0x00E0	#LATIN SMALL LETTER A WITH GRAVE
0xE1	0x00E1	#LATIN SMALL LETTER A WITH ACUTE
0xE2	0x00E2	#LATIN SMALL LETTER A WITH CIRCUMFLEX
0xE3	0x00E3	#LATIN SMALL LETTER A WITH TILDE
0xE4	0x00E4	#LATIN SMALL LETTER A WITH DIAERESIS
0xE5	0x00E5	#LATIN SMALL LETTER A WITH RING ABOVE
0xE6	0x00E6	#LATIN SMALL LETTER AE
0xE7	0x00E7	#LATIN SMALL LETTER C WITH CEDILLA
0xE8	0x00E8	#LATIN SMALL LETTER E WITH GRAVE
0xE9	0x00E9	#LATIN SMALL LETTER E WITH ACUTE
0xEA	0x00EA	#LATIN SMALL LETTER E WITH CIRCUMFLEX
0xEB	0x00EB	#LATIN SMALL LETTER E WITH DIAERESIS
0xEC	0x00EC	#LATIN SMALL LETTER I WITH GRAVE
0xED	0x00ED	#LATIN SMALL LETTER I WITH ACUTE
0xEE	0x00EE	#LATIN SMALL LETTER I WITH CIRCUMFLEX
0xEF	0x00EF	#LATIN SMALL LETTER I WITH DIAERESIS
0xF0	0x011F	#LATIN SMALL LETTER G WITH BREVE
0xF1	0x00F1	#LATIN SMALL LETTER N WITH TILDE
0xF2	0x00F2	#LATIN SMALL LETTER O WITH GRAVE
0xF3	0x00F3	#LATIN SMALL LETTER O WITH ACUTE
0xF4	0x00F4	#LATIN SMALL LETTER O WITH CIRCUMFLEX
0xF5	0x00F5	#LATIN SMALL LETTER O WITH TILDE
0xF6	0x00F6	#LATIN SMALL LETTER O WITH DIAERESIS
0xF7	0x00F7	#DIVISION SIGN
0xF8	0x00F8	#LATIN SMALL LETTER O WITH STROKE
0xF9	0x00F9	#LATIN SMALL LETTER U WITH GRAVE
0xFA	0x00FA	#LATIN SMALL LETTER U WITH ACUTE
0xFB	0x00FB	#LATIN SMALL LETTER U WITH CIRCUMFLEX
0xFC	0x00FC	#LATIN SMALL LETTER U WITH DIAERESIS
0xFD	0x0131	#LATIN SMALL LETTER DOTLESS I
0xFE	0x015F	#LATIN SMALL LETTER S WITH CEDILLA
0xFF	0x00FF	#LATIN SMALL LETTER Y WITH DIAERESIS

tools/catdoc/root/usr/share/catdoc/cp1255.txt

```
#
# Name:      cp1255 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      1/7/2000
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#       Column #1 is the cp1255 code (in hex)
#       Column #2 is the Unicode (in hex as 0xXXXX)
#       Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1255 order
#
0x00    0x0000    #NULL
0x01    0x0001    #START OF HEADING
0x02    0x0002    #START OF TEXT
0x03    0x0003    #END OF TEXT
0x04    0x0004    #END OF TRANSMISSION
0x05    0x0005    #ENQUIRY
0x06    0x0006    #ACKNOWLEDGE
0x07    0x0007    #BELL
0x08    0x0008    #BACKSPACE
0x09    0x0009    #HORIZONTAL TABULATION
0x0A    0x000A    #LINE FEED
0x0B    0x000B    #VERTICAL TABULATION
0x0C    0x000C    #FORM FEED
```

0x0D	0x000D	#CARRIAGE RETURN
0x0E	0x000E	#SHIFT OUT
0x0F	0x000F	#SHIFT IN
0x10	0x0010	#DATA LINK ESCAPE
0x11	0x0011	#DEVICE CONTROL ONE
0x12	0x0012	#DEVICE CONTROL TWO
0x13	0x0013	#DEVICE CONTROL THREE
0x14	0x0014	#DEVICE CONTROL FOUR
0x15	0x0015	#NEGATIVE ACKNOWLEDGE
0x16	0x0016	#SYNCHRONOUS IDLE
0x17	0x0017	#END OF TRANSMISSION BLOCK
0x18	0x0018	#CANCEL
0x19	0x0019	#END OF MEDIUM
0x1A	0x001A	#SUBSTITUTE
0x1B	0x001B	#ESCAPE
0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S

0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88	0x02C6	#MODIFIER LETTER CIRCUMFLEX ACCENT
0x89	0x2030	#PER MILLE SIGN
0x8A		#UNDEFINED
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C		#UNDEFINED
0x8D		#UNDEFINED
0x8E		#UNDEFINED
0x8F		#UNDEFINED
0x90		#UNDEFINED
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98	0x02DC	#SMALL TILDE
0x99	0x2122	#TRADE MARK SIGN
0x9A		#UNDEFINED

0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C		#UNDEFINED
0x9D		#UNDEFINED
0x9E		#UNDEFINED
0x9F		#UNDEFINED
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x00A1	#INVERTED EXCLAMATION MARK
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x20AA	#NEW SHEQEL SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x00D7	#MULTIPLICATION SIGN
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00AF	#MACRON
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERSCRIFT TWO
0xB3	0x00B3	#SUPERSCRIFT THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00B8	#CEDILLA
0xB9	0x00B9	#SUPERSCRIFT ONE
0xBA	0x00F7	#DIVISION SIGN
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x00BF	#INVERTED QUESTION MARK
0xC0	0x05B0	#HEBREW POINT SHEVA
0xC1	0x05B1	#HEBREW POINT HATAF SEGOL
0xC2	0x05B2	#HEBREW POINT HATAF PATAH
0xC3	0x05B3	#HEBREW POINT HATAF QAMATS
0xC4	0x05B4	#HEBREW POINT HIRIQ
0xC5	0x05B5	#HEBREW POINT TSERE
0xC6	0x05B6	#HEBREW POINT SEGOL
0xC7	0x05B7	#HEBREW POINT PATAH
0xC8	0x05B8	#HEBREW POINT QAMATS
0xC9	0x05B9	#HEBREW POINT HOLAM
0xCA		#UNDEFINED
0xCB	0x05BB	#HEBREW POINT QUBUTS
0xCC	0x05BC	#HEBREW POINT DAGESH OR MAPIQ
0xCD	0x05BD	#HEBREW POINT METEG
0xCE	0x05BE	#HEBREW PUNCTUATION MAQAF
0xCF	0x05BF	#HEBREW POINT RAFE
0xD0	0x05C0	#HEBREW PUNCTUATION PASEQ
0xD1	0x05C1	#HEBREW POINT SHIN DOT
0xD2	0x05C2	#HEBREW POINT SIN DOT
0xD3	0x05C3	#HEBREW PUNCTUATION SOF PASUQ
0xD4	0x05F0	#HEBREW LIGATURE YIDDISH DOUBLE VAV
0xD5	0x05F1	#HEBREW LIGATURE YIDDISH VAV YOD
0xD6	0x05F2	#HEBREW LIGATURE YIDDISH DOUBLE YOD
0xD7	0x05F3	#HEBREW PUNCTUATION GERESH
0xD8	0x05F4	#HEBREW PUNCTUATION GERSHAYIM
0xD9		#UNDEFINED
0xDA		#UNDEFINED
0xDB		#UNDEFINED
0xDC		#UNDEFINED
0xDD		#UNDEFINED
0xDE		#UNDEFINED
0xDF		#UNDEFINED
0xE0	0x05D0	#HEBREW LETTER ALEF
0xE1	0x05D1	#HEBREW LETTER BET

0xE2	0x05D2	#HEBREW LETTER GIMEL
0xE3	0x05D3	#HEBREW LETTER DALET
0xE4	0x05D4	#HEBREW LETTER HE
0xE5	0x05D5	#HEBREW LETTER VAV
0xE6	0x05D6	#HEBREW LETTER ZAYIN
0xE7	0x05D7	#HEBREW LETTER HET
0xE8	0x05D8	#HEBREW LETTER TET
0xE9	0x05D9	#HEBREW LETTER YOD
0xEA	0x05DA	#HEBREW LETTER FINAL KAF
0xEB	0x05DB	#HEBREW LETTER KAF
0xEC	0x05DC	#HEBREW LETTER LAMED
0xED	0x05DD	#HEBREW LETTER FINAL MEM
0xEE	0x05DE	#HEBREW LETTER MEM
0xEF	0x05DF	#HEBREW LETTER FINAL NUN
0xF0	0x05E0	#HEBREW LETTER NUN
0xF1	0x05E1	#HEBREW LETTER SAMEKH
0xF2	0x05E2	#HEBREW LETTER AYIN
0xF3	0x05E3	#HEBREW LETTER FINAL PE
0xF4	0x05E4	#HEBREW LETTER PE
0xF5	0x05E5	#HEBREW LETTER FINAL TSADI
0xF6	0x05E6	#HEBREW LETTER TSADI
0xF7	0x05E7	#HEBREW LETTER QOF
0xF8	0x05E8	#HEBREW LETTER RESH
0xF9	0x05E9	#HEBREW LETTER SHIN
0xFA	0x05EA	#HEBREW LETTER TAV
0xFB		#UNDEFINED
0xFC		#UNDEFINED
0xFD	0x200E	#LEFT-TO-RIGHT MARK
0xFE	0x200F	#RIGHT-TO-LEFT MARK
0xFF		#UNDEFINED

tools/catdoc/root/usr/share/catdoc/cp1256.txt

```
#
# Name:      cp1256 to Unicode table
# Unicode version: 2.1
# Table version: 2.01
# Table format: Format A
# Date:      01/5/99
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1256 code (in hex)
#      Column #2 is the Unicode (in hex as 0xXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1256 order
#
0x00  0x0000  #NULL
0x01  0x0001  #START OF HEADING
0x02  0x0002  #START OF TEXT
0x03  0x0003  #END OF TEXT
0x04  0x0004  #END OF TRANSMISSION
0x05  0x0005  #ENQUIRY
0x06  0x0006  #ACKNOWLEDGE
0x07  0x0007  #BELL
0x08  0x0008  #BACKSPACE
0x09  0x0009  #HORIZONTAL TABULATION
0x0A  0x000A  #LINE FEED
0x0B  0x000B  #VERTICAL TABULATION
0x0C  0x000C  #FORM FEED
0x0D  0x000D  #CARRIAGE RETURN
0x0E  0x000E  #SHIFT OUT
0x0F  0x000F  #SHIFT IN
0x10  0x0010  #DATA LINK ESCAPE
0x11  0x0011  #DEVICE CONTROL ONE
```


0x12	0x0012	#DEVICE CONTROL TWO
0x13	0x0013	#DEVICE CONTROL THREE
0x14	0x0014	#DEVICE CONTROL FOUR
0x15	0x0015	#NEGATIVE ACKNOWLEDGE
0x16	0x0016	#SYNCHRONOUS IDLE
0x17	0x0017	#END OF TRANSMISSION BLOCK
0x18	0x0018	#CANCEL
0x19	0x0019	#END OF MEDIUM
0x1A	0x001A	#SUBSTITUTE
0x1B	0x001B	#ESCAPE
0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X

0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81	0x067E	#ARABIC LETTER PEH
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88	0x02C6	#MODIFIER LETTER CIRCUMFLEX ACCENT
0x89	0x2030	#PER MILLE SIGN
0x8A	0x0679	#ARABIC LETTER TTEH
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C	0x0152	#LATIN CAPITAL LIGATURE OE
0x8D	0x0686	#ARABIC LETTER TCHEH
0x8E	0x0698	#ARABIC LETTER JEH
0x8F	0x0688	#ARABIC LETTER DDAL
0x90	0x06AF	#ARABIC LETTER GAF
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98	0x06A9	#ARABIC LETTER KEHEH
0x99	0x2122	#TRADE MARK SIGN
0x9A	0x0691	#ARABIC LETTER RREH
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C	0x0153	#LATIN SMALL LIGATURE OE
0x9D	0x200C	#ZERO WIDTH NON-JOINER
0x9E	0x200D	#ZERO WIDTH JOINER
0x9F	0x06BA	#ARABIC LETTER NOON GHUNNA

0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x060C	#ARABIC COMMA
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x06BE	#ARABIC LETTER HEH DOACHASHMEE
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00AF	#MACRON
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERSCRIT TWO
0xB3	0x00B3	#SUPERSCRIT THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00B8	#CEDILLA
0xB9	0x00B9	#SUPERSCRIT ONE
0xBA	0x061B	#ARABIC SEMICOLON
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x061F	#ARABIC QUESTION MARK
0xC0	0x06C1	#ARABIC LETTER HEH GOAL
0xC1	0x0621	#ARABIC LETTER HAMZA
0xC2	0x0622	#ARABIC LETTER ALEF WITH MADDA ABOVE
0xC3	0x0623	#ARABIC LETTER ALEF WITH HAMZA ABOVE
0xC4	0x0624	#ARABIC LETTER WAW WITH HAMZA ABOVE
0xC5	0x0625	#ARABIC LETTER ALEF WITH HAMZA BELOW
0xC6	0x0626	#ARABIC LETTER YEH WITH HAMZA ABOVE
0xC7	0x0627	#ARABIC LETTER ALEF
0xC8	0x0628	#ARABIC LETTER BEH
0xC9	0x0629	#ARABIC LETTER TEH MARBUTA
0xCA	0x062A	#ARABIC LETTER TEH
0xCB	0x062B	#ARABIC LETTER THEH
0xCC	0x062C	#ARABIC LETTER JEEM
0xCD	0x062D	#ARABIC LETTER HAH
0xCE	0x062E	#ARABIC LETTER KHAH
0xCF	0x062F	#ARABIC LETTER DAL
0xD0	0x0630	#ARABIC LETTER THAL
0xD1	0x0631	#ARABIC LETTER REH
0xD2	0x0632	#ARABIC LETTER ZAIN
0xD3	0x0633	#ARABIC LETTER SEEN
0xD4	0x0634	#ARABIC LETTER SHEEN
0xD5	0x0635	#ARABIC LETTER SAD
0xD6	0x0636	#ARABIC LETTER DAD
0xD7	0x00D7	#MULTIPLICATION SIGN
0xD8	0x0637	#ARABIC LETTER TAH
0xD9	0x0638	#ARABIC LETTER ZAH
0xDA	0x0639	#ARABIC LETTER AIN
0xDB	0x063A	#ARABIC LETTER GHAIN
0xDC	0x0640	#ARABIC TATWEEL
0xDD	0x0641	#ARABIC LETTER FEH
0xDE	0x0642	#ARABIC LETTER QAF
0xDF	0x0643	#ARABIC LETTER KAF
0xE0	0x00E0	#LATIN SMALL LETTER A WITH GRAVE
0xE1	0x0644	#ARABIC LETTER LAM
0xE2	0x00E2	#LATIN SMALL LETTER A WITH CIRCUMFLEX
0xE3	0x0645	#ARABIC LETTER MEEM
0xE4	0x0646	#ARABIC LETTER NOON
0xE5	0x0647	#ARABIC LETTER HEH
0xE6	0x0648	#ARABIC LETTER WAW

0xE7	0x00E7	#LATIN SMALL LETTER C WITH CEDILLA
0xE8	0x00E8	#LATIN SMALL LETTER E WITH GRAVE
0xE9	0x00E9	#LATIN SMALL LETTER E WITH ACUTE
0xEA	0x00EA	#LATIN SMALL LETTER E WITH CIRCUMFLEX
0xEB	0x00EB	#LATIN SMALL LETTER E WITH DIAERESIS
0xEC	0x0649	#ARABIC LETTER ALEF MAKSURA
0xED	0x064A	#ARABIC LETTER YEH
0xEE	0x00EE	#LATIN SMALL LETTER I WITH CIRCUMFLEX
0xEF	0x00EF	#LATIN SMALL LETTER I WITH DIAERESIS
0xF0	0x064B	#ARABIC FATHATAN
0xF1	0x064C	#ARABIC DAMMATAN
0xF2	0x064D	#ARABIC KASRATAN
0xF3	0x064E	#ARABIC FATHA
0xF4	0x00F4	#LATIN SMALL LETTER O WITH CIRCUMFLEX
0xF5	0x064F	#ARABIC DAMMA
0xF6	0x0650	#ARABIC KASRA
0xF7	0x00F7	#DIVISION SIGN
0xF8	0x0651	#ARABIC SHADDA
0xF9	0x00F9	#LATIN SMALL LETTER U WITH GRAVE
0xFA	0x0652	#ARABIC SUKUN
0xFB	0x00FB	#LATIN SMALL LETTER U WITH CIRCUMFLEX
0xFC	0x00FC	#LATIN SMALL LETTER U WITH DIAERESIS
0xFD	0x200E	#LEFT-TO-RIGHT MARK
0xFE	0x200F	#RIGHT-TO-LEFT MARK
0xFF	0x06D2	#ARABIC LETTER YEH BARREE

tools/catdoc/root/usr/share/catdoc/cp1257.txt

```
#
# Name:      cp1257 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      04/15/98
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1257 code (in hex)
#      Column #2 is the Unicode (in hex as 0xXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1257 order
#
0x00      0x0000  #NULL
0x01      0x0001  #START OF HEADING
0x02      0x0002  #START OF TEXT
0x03      0x0003  #END OF TEXT
0x04      0x0004  #END OF TRANSMISSION
0x05      0x0005  #ENQUIRY
0x06      0x0006  #ACKNOWLEDGE
0x07      0x0007  #BELL
0x08      0x0008  #BACKSPACE
0x09      0x0009  #HORIZONTAL TABULATION
0x0A      0x000A  #LINE FEED
0x0B      0x000B  #VERTICAL TABULATION
0x0C      0x000C  #FORM FEED
0x0D      0x000D  #CARRIAGE RETURN
0x0E      0x000E  #SHIFT OUT
0x0F      0x000F  #SHIFT IN
0x10      0x0010  #DATA LINK ESCAPE
0x11      0x0011  #DEVICE CONTROL ONE
0x12      0x0012  #DEVICE CONTROL TWO
0x13      0x0013  #DEVICE CONTROL THREE
0x14      0x0014  #DEVICE CONTROL FOUR
0x15      0x0015  #NEGATIVE ACKNOWLEDGE
0x16      0x0016  #SYNCHRONOUS IDLE
```

0x17	0x0017	#END OF TRANSMISSION BLOCK
0x18	0x0018	#CANCEL
0x19	0x0019	#END OF MEDIUM
0x1A	0x001A	#SUBSTITUTE
0x1B	0x001B	#ESCAPE
0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET

0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B
0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83		#UNDEFINED
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88		#UNDEFINED
0x89	0x2030	#PER MILLE SIGN
0x8A		#UNDEFINED
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C		#UNDEFINED
0x8D	0x00A8	#DIAERESIS
0x8E	0x02C7	#CARON
0x8F	0x00B8	#CEDILLA
0x90		#UNDEFINED
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98		#UNDEFINED
0x99	0x2122	#TRADE MARK SIGN
0x9A		#UNDEFINED
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C		#UNDEFINED
0x9D	0x00AF	#MACRON
0x9E	0x02DB	#OGONEK
0x9F		#UNDEFINED
0xA0	0x00A0	#NO-BREAK SPACE
0xA1		#UNDEFINED
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN

0xA5		#UNDEFINED
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00D8	#LATIN CAPITAL LETTER O WITH STROKE
0xA9	0x00A9	#COPYRIGHT SIGN
0xAA	0x0156	#LATIN CAPITAL LETTER R WITH CEDILLA
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00C6	#LATIN CAPITAL LETTER AE
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERSCRIP T TWO
0xB3	0x00B3	#SUPERSCRIP T THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00F8	#LATIN SMALL LETTER O WITH STROKE
0xB9	0x00B9	#SUPERSCRIP T ONE
0xBA	0x0157	#LATIN SMALL LETTER R WITH CEDILLA
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x00E6	#LATIN SMALL LETTER AE
0xC0	0x0104	#LATIN CAPITAL LETTER A WITH OGONEK
0xC1	0x012E	#LATIN CAPITAL LETTER I WITH OGONEK
0xC2	0x0100	#LATIN CAPITAL LETTER A WITH MACRON
0xC3	0x0106	#LATIN CAPITAL LETTER C WITH ACUTE
0xC4	0x00C4	#LATIN CAPITAL LETTER A WITH DIAERESIS
0xC5	0x00C5	#LATIN CAPITAL LETTER A WITH RING ABOVE
0xC6	0x0118	#LATIN CAPITAL LETTER E WITH OGONEK
0xC7	0x0112	#LATIN CAPITAL LETTER E WITH MACRON
0xC8	0x010C	#LATIN CAPITAL LETTER C WITH CARON
0xC9	0x00C9	#LATIN CAPITAL LETTER E WITH ACUTE
0xCA	0x0179	#LATIN CAPITAL LETTER Z WITH ACUTE
0xCB	0x0116	#LATIN CAPITAL LETTER E WITH DOT ABOVE
0xCC	0x0122	#LATIN CAPITAL LETTER G WITH CEDILLA
0xCD	0x0136	#LATIN CAPITAL LETTER K WITH CEDILLA
0xCE	0x012A	#LATIN CAPITAL LETTER I WITH MACRON
0xCF	0x013B	#LATIN CAPITAL LETTER L WITH CEDILLA
0xD0	0x0160	#LATIN CAPITAL LETTER S WITH CARON
0xD1	0x0143	#LATIN CAPITAL LETTER N WITH ACUTE
0xD2	0x0145	#LATIN CAPITAL LETTER N WITH CEDILLA
0xD3	0x00D3	#LATIN CAPITAL LETTER O WITH ACUTE
0xD4	0x014C	#LATIN CAPITAL LETTER O WITH MACRON
0xD5	0x00D5	#LATIN CAPITAL LETTER O WITH TILDE
0xD6	0x00D6	#LATIN CAPITAL LETTER O WITH DIAERESIS
0xD7	0x00D7	#MULTIPLICATION SIGN
0xD8	0x0172	#LATIN CAPITAL LETTER U WITH OGONEK
0xD9	0x0141	#LATIN CAPITAL LETTER L WITH STROKE
0xDA	0x015A	#LATIN CAPITAL LETTER S WITH ACUTE
0xDB	0x016A	#LATIN CAPITAL LETTER U WITH MACRON
0xDC	0x00DC	#LATIN CAPITAL LETTER U WITH DIAERESIS
0xDD	0x017B	#LATIN CAPITAL LETTER Z WITH DOT ABOVE
0xDE	0x017D	#LATIN CAPITAL LETTER Z WITH CARON
0xDF	0x00DF	#LATIN SMALL LETTER SHARP S
0xE0	0x0105	#LATIN SMALL LETTER A WITH OGONEK
0xE1	0x012F	#LATIN SMALL LETTER I WITH OGONEK
0xE2	0x0101	#LATIN SMALL LETTER A WITH MACRON
0xE3	0x0107	#LATIN SMALL LETTER C WITH ACUTE
0xE4	0x00E4	#LATIN SMALL LETTER A WITH DIAERESIS
0xE5	0x00E5	#LATIN SMALL LETTER A WITH RING ABOVE
0xE6	0x0119	#LATIN SMALL LETTER E WITH OGONEK
0xE7	0x0113	#LATIN SMALL LETTER E WITH MACRON
0xE8	0x010D	#LATIN SMALL LETTER C WITH CARON
0xE9	0x00E9	#LATIN SMALL LETTER E WITH ACUTE
0xEA	0x017A	#LATIN SMALL LETTER Z WITH ACUTE
0xEB	0x0117	#LATIN SMALL LETTER E WITH DOT ABOVE

0xEC	0x0123	#LATIN SMALL LETTER G WITH CEDILLA
0xED	0x0137	#LATIN SMALL LETTER K WITH CEDILLA
0xEE	0x012B	#LATIN SMALL LETTER I WITH MACRON
0xEF	0x013C	#LATIN SMALL LETTER L WITH CEDILLA
0xF0	0x0161	#LATIN SMALL LETTER S WITH CARON
0xF1	0x0144	#LATIN SMALL LETTER N WITH ACUTE
0xF2	0x0146	#LATIN SMALL LETTER N WITH CEDILLA
0xF3	0x00F3	#LATIN SMALL LETTER O WITH ACUTE
0xF4	0x014D	#LATIN SMALL LETTER O WITH MACRON
0xF5	0x00F5	#LATIN SMALL LETTER O WITH TILDE
0xF6	0x00F6	#LATIN SMALL LETTER O WITH DIAERESIS
0xF7	0x00F7	#DIVISION SIGN
0xF8	0x0173	#LATIN SMALL LETTER U WITH OGONEK
0xF9	0x0142	#LATIN SMALL LETTER L WITH STROKE
0xFA	0x015B	#LATIN SMALL LETTER S WITH ACUTE
0xFB	0x016B	#LATIN SMALL LETTER U WITH MACRON
0xFC	0x00FC	#LATIN SMALL LETTER U WITH DIAERESIS
0xFD	0x017C	#LATIN SMALL LETTER Z WITH DOT ABOVE
0xFE	0x017E	#LATIN SMALL LETTER Z WITH CARON
0xFF	0x02D9	#DOT ABOVE

tools/catdoc/root/usr/share/catdoc/cp1258.txt

```
#
# Name:      cp1258 to Unicode table
# Unicode version: 2.0
# Table version: 2.01
# Table format: Format A
# Date:      04/15/98
#
# Contact:    Shawn.Steele@microsoft.com
#
# General notes: none
#
# Format: Three tab-separated columns
#      Column #1 is the cp1258 code (in hex)
#      Column #2 is the Unicode (in hex as 0xXXXX)
#      Column #3 is the Unicode name (follows a comment sign, '#')
#
# The entries are in cp1258 order
#
0x00 0x0000 #NULL
0x01 0x0001 #START OF HEADING
0x02 0x0002 #START OF TEXT
0x03 0x0003 #END OF TEXT
0x04 0x0004 #END OF TRANSMISSION
0x05 0x0005 #ENQUIRY
0x06 0x0006 #ACKNOWLEDGE
0x07 0x0007 #BELL
0x08 0x0008 #BACKSPACE
0x09 0x0009 #HORIZONTAL TABULATION
0x0A 0x000A #LINE FEED
0x0B 0x000B #VERTICAL TABULATION
0x0C 0x000C #FORM FEED
0x0D 0x000D #CARRIAGE RETURN
0x0E 0x000E #SHIFT OUT
0x0F 0x000F #SHIFT IN
0x10 0x0010 #DATA LINK ESCAPE
0x11 0x0011 #DEVICE CONTROL ONE
0x12 0x0012 #DEVICE CONTROL TWO
0x13 0x0013 #DEVICE CONTROL THREE
0x14 0x0014 #DEVICE CONTROL FOUR
0x15 0x0015 #NEGATIVE ACKNOWLEDGE
0x16 0x0016 #SYNCHRONOUS IDLE
0x17 0x0017 #END OF TRANSMISSION BLOCK
0x18 0x0018 #CANCEL
0x19 0x0019 #END OF MEDIUM
0x1A 0x001A #SUBSTITUTE
0x1B 0x001B #ESCAPE
```


0x1C	0x001C	#FILE SEPARATOR
0x1D	0x001D	#GROUP SEPARATOR
0x1E	0x001E	#RECORD SEPARATOR
0x1F	0x001F	#UNIT SEPARATOR
0x20	0x0020	#SPACE
0x21	0x0021	#EXCLAMATION MARK
0x22	0x0022	#QUOTATION MARK
0x23	0x0023	#NUMBER SIGN
0x24	0x0024	#DOLLAR SIGN
0x25	0x0025	#PERCENT SIGN
0x26	0x0026	#AMPERSAND
0x27	0x0027	#APOSTROPHE
0x28	0x0028	#LEFT PARENTHESIS
0x29	0x0029	#RIGHT PARENTHESIS
0x2A	0x002A	#ASTERISK
0x2B	0x002B	#PLUS SIGN
0x2C	0x002C	#COMMA
0x2D	0x002D	#HYPHEN-MINUS
0x2E	0x002E	#FULL STOP
0x2F	0x002F	#SOLIDUS
0x30	0x0030	#DIGIT ZERO
0x31	0x0031	#DIGIT ONE
0x32	0x0032	#DIGIT TWO
0x33	0x0033	#DIGIT THREE
0x34	0x0034	#DIGIT FOUR
0x35	0x0035	#DIGIT FIVE
0x36	0x0036	#DIGIT SIX
0x37	0x0037	#DIGIT SEVEN
0x38	0x0038	#DIGIT EIGHT
0x39	0x0039	#DIGIT NINE
0x3A	0x003A	#COLON
0x3B	0x003B	#SEMICOLON
0x3C	0x003C	#LESS-THAN SIGN
0x3D	0x003D	#EQUALS SIGN
0x3E	0x003E	#GREATER-THAN SIGN
0x3F	0x003F	#QUESTION MARK
0x40	0x0040	#COMMERCIAL AT
0x41	0x0041	#LATIN CAPITAL LETTER A
0x42	0x0042	#LATIN CAPITAL LETTER B
0x43	0x0043	#LATIN CAPITAL LETTER C
0x44	0x0044	#LATIN CAPITAL LETTER D
0x45	0x0045	#LATIN CAPITAL LETTER E
0x46	0x0046	#LATIN CAPITAL LETTER F
0x47	0x0047	#LATIN CAPITAL LETTER G
0x48	0x0048	#LATIN CAPITAL LETTER H
0x49	0x0049	#LATIN CAPITAL LETTER I
0x4A	0x004A	#LATIN CAPITAL LETTER J
0x4B	0x004B	#LATIN CAPITAL LETTER K
0x4C	0x004C	#LATIN CAPITAL LETTER L
0x4D	0x004D	#LATIN CAPITAL LETTER M
0x4E	0x004E	#LATIN CAPITAL LETTER N
0x4F	0x004F	#LATIN CAPITAL LETTER O
0x50	0x0050	#LATIN CAPITAL LETTER P
0x51	0x0051	#LATIN CAPITAL LETTER Q
0x52	0x0052	#LATIN CAPITAL LETTER R
0x53	0x0053	#LATIN CAPITAL LETTER S
0x54	0x0054	#LATIN CAPITAL LETTER T
0x55	0x0055	#LATIN CAPITAL LETTER U
0x56	0x0056	#LATIN CAPITAL LETTER V
0x57	0x0057	#LATIN CAPITAL LETTER W
0x58	0x0058	#LATIN CAPITAL LETTER X
0x59	0x0059	#LATIN CAPITAL LETTER Y
0x5A	0x005A	#LATIN CAPITAL LETTER Z
0x5B	0x005B	#LEFT SQUARE BRACKET
0x5C	0x005C	#REVERSE SOLIDUS
0x5D	0x005D	#RIGHT SQUARE BRACKET
0x5E	0x005E	#CIRCUMFLEX ACCENT
0x5F	0x005F	#LOW LINE
0x60	0x0060	#GRAVE ACCENT
0x61	0x0061	#LATIN SMALL LETTER A
0x62	0x0062	#LATIN SMALL LETTER B

0x63	0x0063	#LATIN SMALL LETTER C
0x64	0x0064	#LATIN SMALL LETTER D
0x65	0x0065	#LATIN SMALL LETTER E
0x66	0x0066	#LATIN SMALL LETTER F
0x67	0x0067	#LATIN SMALL LETTER G
0x68	0x0068	#LATIN SMALL LETTER H
0x69	0x0069	#LATIN SMALL LETTER I
0x6A	0x006A	#LATIN SMALL LETTER J
0x6B	0x006B	#LATIN SMALL LETTER K
0x6C	0x006C	#LATIN SMALL LETTER L
0x6D	0x006D	#LATIN SMALL LETTER M
0x6E	0x006E	#LATIN SMALL LETTER N
0x6F	0x006F	#LATIN SMALL LETTER O
0x70	0x0070	#LATIN SMALL LETTER P
0x71	0x0071	#LATIN SMALL LETTER Q
0x72	0x0072	#LATIN SMALL LETTER R
0x73	0x0073	#LATIN SMALL LETTER S
0x74	0x0074	#LATIN SMALL LETTER T
0x75	0x0075	#LATIN SMALL LETTER U
0x76	0x0076	#LATIN SMALL LETTER V
0x77	0x0077	#LATIN SMALL LETTER W
0x78	0x0078	#LATIN SMALL LETTER X
0x79	0x0079	#LATIN SMALL LETTER Y
0x7A	0x007A	#LATIN SMALL LETTER Z
0x7B	0x007B	#LEFT CURLY BRACKET
0x7C	0x007C	#VERTICAL LINE
0x7D	0x007D	#RIGHT CURLY BRACKET
0x7E	0x007E	#TILDE
0x7F	0x007F	#DELETE
0x80	0x20AC	#EURO SIGN
0x81		#UNDEFINED
0x82	0x201A	#SINGLE LOW-9 QUOTATION MARK
0x83	0x0192	#LATIN SMALL LETTER F WITH HOOK
0x84	0x201E	#DOUBLE LOW-9 QUOTATION MARK
0x85	0x2026	#HORIZONTAL ELLIPSIS
0x86	0x2020	#DAGGER
0x87	0x2021	#DOUBLE DAGGER
0x88	0x02C6	#MODIFIER LETTER CIRCUMFLEX ACCENT
0x89	0x2030	#PER MILLE SIGN
0x8A		#UNDEFINED
0x8B	0x2039	#SINGLE LEFT-POINTING ANGLE QUOTATION MARK
0x8C	0x0152	#LATIN CAPITAL LIGATURE OE
0x8D		#UNDEFINED
0x8E		#UNDEFINED
0x8F		#UNDEFINED
0x90		#UNDEFINED
0x91	0x2018	#LEFT SINGLE QUOTATION MARK
0x92	0x2019	#RIGHT SINGLE QUOTATION MARK
0x93	0x201C	#LEFT DOUBLE QUOTATION MARK
0x94	0x201D	#RIGHT DOUBLE QUOTATION MARK
0x95	0x2022	#BULLET
0x96	0x2013	#EN DASH
0x97	0x2014	#EM DASH
0x98	0x02DC	#SMALL TILDE
0x99	0x2122	#TRADE MARK SIGN
0x9A		#UNDEFINED
0x9B	0x203A	#SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
0x9C	0x0153	#LATIN SMALL LIGATURE OE
0x9D		#UNDEFINED
0x9E		#UNDEFINED
0x9F	0x0178	#LATIN CAPITAL LETTER Y WITH DIAERESIS
0xA0	0x00A0	#NO-BREAK SPACE
0xA1	0x00A1	#INVERTED EXCLAMATION MARK
0xA2	0x00A2	#CENT SIGN
0xA3	0x00A3	#POUND SIGN
0xA4	0x00A4	#CURRENCY SIGN
0xA5	0x00A5	#YEN SIGN
0xA6	0x00A6	#BROKEN BAR
0xA7	0x00A7	#SECTION SIGN
0xA8	0x00A8	#DIAERESIS
0xA9	0x00A9	#COPYRIGHT SIGN

0xAA	0x00AA	#FEMININE ORDINAL INDICATOR
0xAB	0x00AB	#LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
0xAC	0x00AC	#NOT SIGN
0xAD	0x00AD	#SOFT HYPHEN
0xAE	0x00AE	#REGISTERED SIGN
0xAF	0x00AF	#MACRON
0xB0	0x00B0	#DEGREE SIGN
0xB1	0x00B1	#PLUS-MINUS SIGN
0xB2	0x00B2	#SUPERScript TWO
0xB3	0x00B3	#SUPERScript THREE
0xB4	0x00B4	#ACUTE ACCENT
0xB5	0x00B5	#MICRO SIGN
0xB6	0x00B6	#PILCROW SIGN
0xB7	0x00B7	#MIDDLE DOT
0xB8	0x00B8	#CEDILLA
0xB9	0x00B9	#SUPERScript ONE
0xBA	0x00BA	#MASCULINE ORDINAL INDICATOR
0xBB	0x00BB	#RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
0xBC	0x00BC	#VULGAR FRACTION ONE QUARTER
0xBD	0x00BD	#VULGAR FRACTION ONE HALF
0xBE	0x00BE	#VULGAR FRACTION THREE QUARTERS
0xBF	0x00BF	#INVERTED QUESTION MARK
0xC0	0x00C0	#LATIN CAPITAL LETTER A WITH GRAVE
0xC1	0x00C1	#LATIN CAPITAL LETTER A WITH ACUTE
0xC2	0x00C2	#LATIN CAPITAL LETTER A WITH CIRCUMFLEX
0xC3	0x0102	#LATIN CAPITAL LETTER A WITH BREVE
0xC4	0x00C4	#LATIN CAPITAL LETTER A WITH DIAERESIS
0xC5	0x00C5	#LATIN CAPITAL LETTER A WITH RING ABOVE
0xC6	0x00C6	#LATIN CAPITAL LETTER AE
0xC7	0x00C7	#LATIN CAPITAL LETTER C WITH CEDILLA
0xC8	0x00C8	#LATIN CAPITAL LETTER E WITH GRAVE
0xC9	0x00C9	#LATIN CAPITAL LETTER E WITH ACUTE
0xCA	0x00CA	#LATIN CAPITAL LETTER E WITH CIRCUMFLEX
0xCB	0x00CB	#LATIN CAPITAL LETTER E WITH DIAERESIS
0xCC	0x0300	#COMBINING GRAVE ACCENT
0xCD	0x00CD	#LATIN CAPITAL LETTER I WITH ACUTE
0xCE	0x00CE	#LATIN CAPITAL LETTER I WITH CIRCUMFLEX
0xCF	0x00CF	#LATIN CAPITAL LETTER I WITH DIAERESIS
0xD0	0x0110	#LATIN CAPITAL LETTER D WITH STROKE
0xD1	0x00D1	#LATIN CAPITAL LETTER N WITH TILDE
0xD2	0x0309	#COMBINING HOOK ABOVE
0xD3	0x00D3	#LATIN CAPITAL LETTER O WITH ACUTE
0xD4	0x00D4	#LATIN CAPITAL LETTER O WITH CIRCUMFLEX
0xD5	0x01A0	#LATIN CAPITAL LETTER O WITH HORN
0xD6	0x00D6	#LATIN CAPITAL LETTER O WITH DIAERESIS
0xD7	0x00D7	#MULTIPLICATION SIGN
0xD8	0x00D8	#LATIN CAPITAL LETTER O WITH STROKE
0xD9	0x00D9	#LATIN CAPITAL LETTER U WITH GRAVE
0xDA	0x00DA	#LATIN CAPITAL LETTER U WITH ACUTE
0xDB	0x00DB	#LATIN CAPITAL LETTER U WITH CIRCUMFLEX
0xDC	0x00DC	#LATIN CAPITAL LETTER U WITH DIAERESIS
0xDD	0x01AF	#LATIN CAPITAL LETTER U WITH HORN
0xDE	0x0303	#COMBINING TILDE
0xDF	0x00DF	#LATIN SMALL LETTER SHARP S
0xE0	0x00E0	#LATIN SMALL LETTER A WITH GRAVE
0xE1	0x00E1	#LATIN SMALL LETTER A WITH ACUTE
0xE2	0x00E2	#LATIN SMALL LETTER A WITH CIRCUMFLEX
0xE3	0x0103	#LATIN SMALL LETTER A WITH BREVE
0xE4	0x00E4	#LATIN SMALL LETTER A WITH DIAERESIS
0xE5	0x00E5	#LATIN SMALL LETTER A WITH RING ABOVE
0xE6	0x00E6	#LATIN SMALL LETTER AE
0xE7	0x00E7	#LATIN SMALL LETTER C WITH CEDILLA
0xE8	0x00E8	#LATIN SMALL LETTER E WITH GRAVE
0xE9	0x00E9	#LATIN SMALL LETTER E WITH ACUTE
0xEA	0x00EA	#LATIN SMALL LETTER E WITH CIRCUMFLEX
0xEB	0x00EB	#LATIN SMALL LETTER E WITH DIAERESIS
0xEC	0x0301	#COMBINING ACUTE ACCENT
0xED	0x00ED	#LATIN SMALL LETTER I WITH ACUTE
0xEE	0x00EE	#LATIN SMALL LETTER I WITH CIRCUMFLEX
0xEF	0x00EF	#LATIN SMALL LETTER I WITH DIAERESIS
0xF0	0x0111	#LATIN SMALL LETTER D WITH STROKE

0xF1	0x00F1	#LATIN SMALL LETTER N WITH TILDE
0xF2	0x0323	#COMBINING DOT BELOW
0xF3	0x00F3	#LATIN SMALL LETTER O WITH ACUTE
0xF4	0x00F4	#LATIN SMALL LETTER O WITH CIRCUMFLEX
0xF5	0x01A1	#LATIN SMALL LETTER O WITH HORN
0xF6	0x00F6	#LATIN SMALL LETTER O WITH DIAERESIS
0xF7	0x00F7	#DIVISION SIGN
0xF8	0x00F8	#LATIN SMALL LETTER O WITH STROKE
0xF9	0x00F9	#LATIN SMALL LETTER U WITH GRAVE
0xFA	0x00FA	#LATIN SMALL LETTER U WITH ACUTE
0xFB	0x00FB	#LATIN SMALL LETTER U WITH CIRCUMFLEX
0xFC	0x00FC	#LATIN SMALL LETTER U WITH DIAERESIS
0xFD	0x01B0	#LATIN SMALL LETTER U WITH HORN
0xFE	0x20AB	#DONG SIGN
0xFF	0x00FF	#LATIN SMALL LETTER Y WITH DIAERESIS